

Practica de Programación Orientada a Objeto – Curs 2025-2026

Contenido

Datos del alumno	3
1. Introducción y planteamiento del problema	3
1.1 Actores participantes.....	3
1.2 Funcionalidades	3
1.3 Relación entre actores	3
2. Análisis de la aplicación.....	4
3. Decisiones de diseño	5
3.1 Organización en paquetes.....	5
3.2 Patrón Observador para el Dashboard.....	5
3.2 Herencias.....	5
3.4 Estado del montaje	6
3.5 Planificador.....	6
3.6 Robots y Operarios.....	6
3.7 Validación de stock previa.....	6
3.8 Historial de operaciones.....	7
4. Diagrama de clases.....	7
5. Descripción de las clases	7
5.1 com.practica.vehiculo	7
5.2 Paquetes de componentes.....	8
5.3 com.practica.personal.....	8
5.5 com.practica.fabrica.....	8
5.6 com.practica.planificador.....	9
5.7 com.practica.dashboard.....	9
5.8 factory_main	10
6. Funcionamiento del programa.....	10
7. AnexoCodigo	13
7.1 com.practica.vehiculo	13
7.2 com.practica.tapiceria.....	22
7.3 com.practica.rueda	27
7.4 com.practica.motor.....	33
7.5 com.practica.personal.....	38

Practica de Programación Orientada a Objeto – Curs 2025-2026

7.6 com.practica.dashboard.....	50
7.7 com.practica.fabrica.....	55
7.8 com.practica.montaje	81
7.9 com.practica.planificador.....	86
7.10 factory_main.java.....	100

Practica de Programación Orientada a Objeto – Curs 2025-2026

Datos del alumno

- Nombre: Xabier Shaoning Capilla Sanz
- Apellidos: Capilla Sanz
- Dirección: Vilariño 37A, Allariz
- Correo: shao.capilla@gmail.com
- Teléfono: 687545767

1. Introducción y planteamiento del problema

La práctica simula una fábrica de vehículos, se divide en 2 unidades operativas: la cadena de montaje y el sistema de gestión de fábrica. En este proyecto se va a aplicar los pilares de la POO de Java.

1.1 Actores participantes

- Operario (eficiente / estándar): controla los robots de montaje.
- Mecánico de cinta (eficiente / estándar): repara averías.
- Gestor de planta: configura las cadenas, monitoriza el dashboard y llama a los mecánicos.
- Administrador del sistema: restaura el sistema y las cadenas tras un apagón.

1.2 Funcionalidades

- Registrar y mantener stock de componentes.
- Encolar vehículos en la cadena.
- Gestionar la plantilla de trabajadores.
- Simular el proceso de montaje tick a tick mediante un planificador con tres niveles de complejidad.
- Mostrar la actividad en un dashboard desacoplado del subistema de visualización.
- Persistir un historial consultable por fecha y producir listados / estadísticas.

1.3 Relación entre actores

- Gestor de Planta -> configura -> Cadena de Montaje
- Cadena de Montaje -> contiene -> Coches en montaje
- Planificador -> orquesta -> Robots -> asignan -> Operarios
- Planificador -> consume -> Stock
- Mecánico -> repara <- (llamado por) <- Gestor Planta
- Almacén / Cadena / Planificador -> notifican -> Dashboard

Practica de Programación Orientada a Objeto – Curs 2025-2026

2. Análisis de la aplicación

La aplicación arranca desde `factory_main`, este ofrece un menú textual con las siguientes opciones:

1. Gestión de Almacén: dar alta los componentes, los vehículos en cadena y modificar atributos de los componentes.
2. Gestión de Trabajadores: dar alta al personal.
3. Consultar stock: visualizar el informe del stock disponibles y los vehículos pendientes/terminados.
4. Buscar empleados: búsqueda por nombre, dni, listar todo el personal de las diferentes categorías.
5. Iniciar Simulación: simple, compleja o muy compleja.
6. Listados y estadísticas: operarios ordenados o filtrados por productividad, vehículos terminados filtrado por componentes, etc.
7. Reiniciar Sistema

El núcleo del sistema lo conforman las siguientes clases:

- `AlmacenDatos` (implementa `IfaceAlmacen`)
- `SistemGestion` (implementa `IfaceAlmacen`)
- `CadenaMontaje`
- `Dashboard` (observador de `AlmacenDatos`, `CadenaMontaje` y `Planificador`)

Practica de Programación Orientada a Objeto – Curs 2025-2026

3. Decisiones de diseño

3.1 Organización en paquetes

El código se estructura en paquetes por responsabilidad.

Paquete	Responsabilidad
com.practica.vehiculo	Jerarquía de vehículos y el enum de estados
com.practica.motor/tapiciera/rueda	Componentes
com.practica.personal	Jerarquía de trabajadores
com.practica.fabrica	Almacén, gestión e historial
com.practica.montaje	Cadena de montajes y robots
com.practica.planificador	Núcleo de la simulación
com.practica.dashboard	Implementa patrón Observer y encargado de la visualización

Justificación:

- Control de visibilidad: me ayuda a tener bien gestionado las diferentes clases que conforman el proyecto para facilitar el trabajo.

3.2 Patrón Observador para el Dashboard

AlmacenDatos, CadenaMontaje y Planificador implementan el interface Observable.

El Dashboard implementa el interface Observador y delega la representación de los datos al interface IfaceVisualizarDatos (lo implementa la Clase VisualizarDatos)

Justificación:

- Cualquier cambio que se realice en el almacén o la cadena, este se propaga al dashboard de manera automática.

3.2 Herencias

Se ha decidido que las clases principales se construirán como clases abstracta ya que no me interesa crear objetos de la clase principal sino sus clases hijas.

- Coche (abstracta) -> BiplazaDeportivo, Turismo, Furgoneta
- Motor (abstracta) -> Gasolina, Electrico, Hibrido
- Tapicera (abstracta) -> Alcantara, Cuero, Tela
- Trabajador (abstracta) -> Operario, Mecanico, AdministradorSistema, GestorPlanta

Practica de Programación Orientada a Objeto – Curs 2025-2026

Cada clase abstracta tiene un método para retornar el tipo que las clases implementan, aplicando el uso de polimorfismo: `tipoCoche()`, `tipoMotor()`, `tipoTapiceria()`, `tipoRueda()`

3.4 Estado del montaje

El enum `EstadoMontaje` cuenta con los estados CHASIS, MOTOR, TAPICERIA, RUEDA, TERMINADO como se indica en el enunciado.

3.5 Planificador

El Planificador actúa como un reloj con `Thread.sleep(1000)`: cada tick avanza vehículos, resuelve incidencias y genera eventos. Soporta los niveles:

Nivel	Averías	Apagón	Personal mínimo
1. Simple	No	No	Operarios
2. Compleja	> 2 por cadena	No	Operarios + Mecánicos
3. Muy Compleja	2-3 por cadena	1 (> 3s)	Operarios + Mecánicos + Administrador

3.6 Robots y Operarios

Cada cadena tiene 4 robots (uno por estación). Cada robot lleva un operario asignado y trabaja a la velocidad que marca su perfil (`getTiempoMontaje()`): 1s eficiente, 3s estándar). Si no hay operarios registrados, Planificador genera operarios para cubrir las 12 estaciones, garantizando que la simulación funcione siempre.

3.7 Validación de stock previa

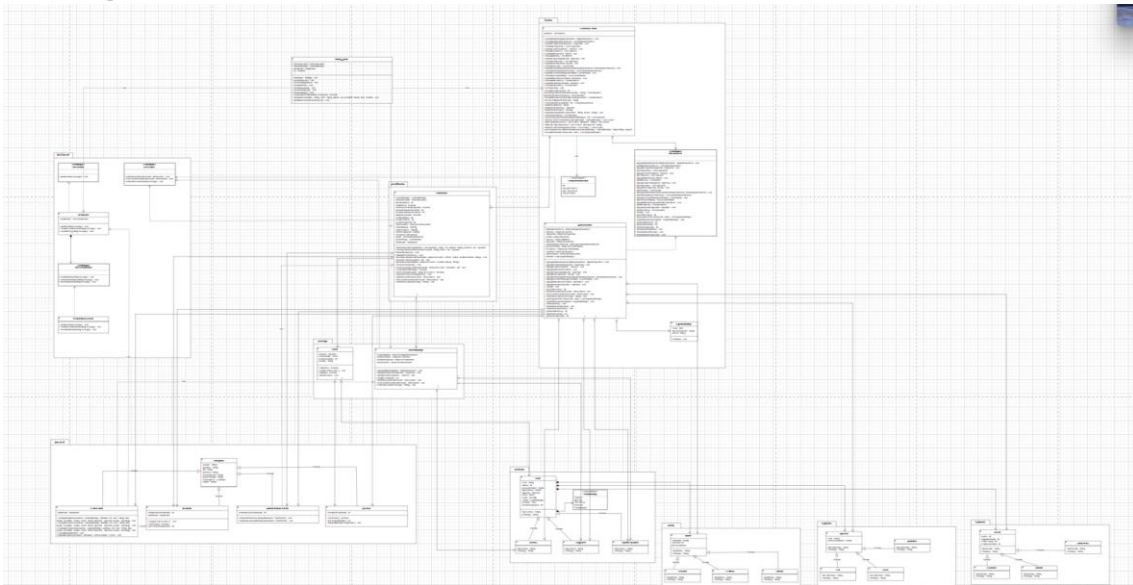
Antes de admitir un nuevo vehículo en la cadena se invoca `comprobarStockParaNuevoVehiculo()`, que llamada a `SistemaGestion.hayStockSuficiente(n+1)` teniendo en cuenta los vehículos ya pendientes. De esta forma nunca se inicia la simulación que no se pueda completar.

Practica de Programación Orientada a Objeto – Curs 2025-2026

3.8 Historial de operaciones

Cada cambio de estado, alta de component o vehículo terminado genera un RegistroMontaje(fecha, tipoComponente, acción) que el almacén guarda. La consulta se realiza por fecha y recupera los registros del día indicado.

4. Diagrama de clases



5. Descripción de las clases

5.1 com.practica.vehiculo

Clase	Tipo	Responsabilidad
Coche	abstracta	Atributos comunes (color, plazas, peso, motor, tapicería, ruedas, estado, avería). Define tipoCoche()
BiplazaDeportivo	concreta	tipoCoche() = "Biplaza Deportivo"
Turismo	concreta	tipoCoche() = "Turismo"
Furgoneta	concreta	tipoCoche() = "Furgoneta"

Practica de Programación Orientada a Objeto – Curs 2025-2026

EstadoMontaje	enum	Estados del proceso: CHASIS, MOTOR, TAPICERIA, RUEDAS, TERMINADO.
---------------	------	---

5.2 Paquetes de componentes

- com.practica.motor: Motor (abstracta) + Gasolina, Electrico, Hibrido.
- com.practica.tapiceria: Tapiceria (abstracta) + Tela, Cuero, Alcantara.
- com.practica.rueda: Rueda (abstracta) + Normal, Deportivo, Todoterreno.

Todos tienen declarado el metodo tipo(nombreClasePadre()) en el que devolverá el tipo de clase correspondiente

5.3 com.practica.personal

Clase	Métodos relevantes
Trabajador (abstracta)	Getters/setters de nombre, apellidos, DNI, dirección, NSS, puesto, salario, fecha
Operario	esEficiente() (>10 montajes), getTiempoMontaje() (1 s / 3 s), registrarMontajeCompletado().
Mecanico	repararCoche(Coche), esEficiente() (>20 reparaciones), getTiempoReparacion() (1 s / 2-5 s).
GestorPlanta	configurarBiplazas/Turismos/Furgonetas(...), consultarDashboard(...), llamarMecanico(...).
AdministrarSistema	restaurarSistemaGestion(Planificador), restaurarCadenasMontaje(Planificador).

5.5 com.practica.fabrica

Clase	Tipo	Responsabilidad
IfaceAlmacen	interfaz	Contrato del almacén (alta y consulta de vehículos, componentes y trabajadores; gestión de stock; historial).
AlmacenDatos	concreta	Implementación basada en ArrayList; además es Observable (notifica al dashboard cada alta de componente o vehículo terminado).
SistemaGestion	concreta	Fachada que opera contra IfaceAlmacen y métodos: registrarXxx, consultarXxx, comprobarStock, quitarStockMotor/Tapiceria/Ruedas,

Practica de Programación Orientada a Objeto – Curs 2025-2026

		buscarOperariosPorNombre/Eficientes, buscarTrabajadorPorDni, ordenarOperarios, obtenerOperariosProductividad, obtenerCochesTerminados, filtrarTipoMotor/Tapiceria, obtenerCochesOrdenados, getConfiguracionesMasEnsamblados, consultarHistorialFecha.
RegistroMontaje	concreta	Entrada del historial (fecha, tipo de componente, acción).
ComprobacionStock	enum	Resultado semántico de la validación de stock previa al encolado de un vehículo. Sus literales (OK, SIN_MOTORES, SIN_TAPICERIAS, SIN_RUEDAS) permiten que SistemaGestion.comprobarStock() comunique exactamente qué componente es insuficiente.

5.6 com.practica.planificador

Planificador: motor de simulación. Construye 12 robots (4 por cadena), recibe los mecánicos y el administrador. iniciarSimulacion() ejecuta el bucle principal: resolver incidencias → trabajar en estaciones → generar eventos → comprobar fin. Soporta los tres niveles. Es Observable.

5.7 com.practica.dashboard

Clase	Tipo	Responsabilidad
Observable / Observador	interfaz	Patrón Observer
Dashboard	concreta	Implementa Observador y delega la presentación en una IfaceVisualizarDatos
IfaceVisualizarDatos	interfaz	Contrato del subsistema de visualización
VisualizarConsola	concreta	Implementación que imprime por System.out.

Practica de Programación Orientada a Objeto – Curs 2025-2026

5.8 factory_main

Punto de entrada. Inicializa el sistema, registra el dashboard como observador y gestiona los seis menús (almacén, trabajadores, stock, búsqueda, simulación, listados). Toda la entrada se realiza por Scanner sobre System.in.

6. Funcionamiento del programa

Flujo de uso

1. Menú Gestion de Almacen: registra los componentes del vehículo.
2. Crear al menos un vehículo, el sistema valida el stok antes de aceptarlo.
3. Menú Gestión de Trabajadores: da de alta operarios, gestor de planta, mecánicos y administrador.
4. Menú Iniciar Simulación: elegir el nivel.
5. Una vez termine la simulación, los vehículos terminados pasan al almacén.
6. Consultar Stock o Listar estadísticas.

GESTIÓN FÁBRICA DE VEHÍCULOS

```
1. Gestión de Almacén
2. Gestión de Trabajadores
3. Consultar stock
4. Buscar empleados
5. Iniciar Simulación
6. Listados y estadísticas
7. Reiniciar Sistema
8. Salir
Elige una opción: 1

MENÚ GESTIÓN DE ALMACÉN
1. Añadir Motor Gasolina
2. Añadir Motor Eléctrico
3. Añadir Motor Híbrido
4. Añadir Tapicería Tela
5. Añadir Tapicería Cuero
6. Añadir Tapicería Alcántara
7. Añadir Rueda Normal
8. Añadir Rueda Deportiva
9. Añadir Rueda Todoterreno
-- VEHÍCULOS A ENSAMBLAR --
10. Añadir Biplaza Deportivo a la cadena
11. Añadir Turismo a la cadena
12. Añadir Furgoneta a la cadena
13. Actualizar componente del almacén
0. Volver al menú principal
Opción:
```

MENÚ GESTIÓN DE ALMACÉN

```
1. Añadir Motor Gasolina
2. Añadir Motor Eléctrico
3. Añadir Motor Híbrido
4. Añadir Tapicería Tela
5. Añadir Tapicería Cuero
6. Añadir Tapicería Alcántara
7. Añadir Rueda Normal
8. Añadir Rueda Deportiva
9. Añadir Rueda Todoterreno
-- VEHÍCULOS A ENSAMBLAR --
10. Añadir Biplaza Deportivo a la cadena
11. Añadir Turismo a la cadena
12. Añadir Furgoneta a la cadena
13. Actualizar componente del almacén
0. Volver al menú principal
Opción: 1
Cilindrada: 1000
Potencia: 300
Cilindros: 600
```

[NOTIFICACIÓN DASHBOARD]

Mensaje: Nuevo componente en almacén, Motor Gasolina

[CONFIRMACIÓN DASHBOARD]

Mensaje: Motor Gasolina registrado.

Practica de Programación Orientada a Objeto – Curs 2025-2026

MENÚ GESTIÓN DE ALMACÉN

1. Añadir Motor Gasolina
2. Añadir Motor Eléctrico
3. Añadir Motor Híbrido
4. Añadir Tapicería Tela
5. Añadir Tapicería Cuero
6. Añadir Tapicería Alcantara
7. Añadir Rueda Normal
8. Añadir Rueda Deportiva
9. Añadir Rueda Todoterreno
- VEHÍCULOS A ENSAMBLAR --
10. Añadir Biplaza Deportivo a la cadena
11. Añadir Turismo a la cadena
12. Añadir Furgoneta a la cadena
13. Actualizar componente del almacén
0. Volver al menú principal

Opción: 11

Color: Rojo

Nº plazas: 3

Peso Autorizado: 1000

Tara Vehículo: 500

[ERROR DASHBOARD]

Mensaje: No se añade el vehículo, stock insuficiente de SIN_TAPICERIAS

GESTIÓN FÁBRICA DE VEHÍCULOS

1. Gestión de Almacén
2. Gestión de Trabajadores
3. Consultar stock
4. Buscar empleados
5. Iniciar Simulación
6. Listados y estadísticas
7. Reiniciar Sistema
0. Salir

Elige una opción: 3

[NOTIFICACIÓN DASHBOARD]

Mensaje: STOCK DEL ALMACÉN

Motores disponibles: 1

Tapicerías disponibles: 1

Ruedas disponibles: 4

VEHÍCULOS EN CADENA (pendientes de ensamblar)

Biplazas: 0

Turismos: 1

Furgonetas: 0

VEHÍCULOS ENSAMBLADOS (histórico)

Biplazas: 0

Turismos: 0

Furgonetas: 0

TOTAL coches: 0

Practica de Programación Orientada a Objeto – Curs 2025-2026

MENÚ GESTIÓN DE TRABAJADORES

1. Añadir Operario
2. Añadir Mecánico
3. Añadir Gestor de Planta
4. Añadir Administrador de Sistema
0. Volver al menú principal

Opción: 1

Nombre: Juan

Apellidos: Perez Diaz

DNI: 1233456K

Dirección: Madrid

Nº Seg. Social: 1

Salario: 40000

[CONFIRMACIÓN DASHBOARD]

Mensaje: Operario registrado.

MENÚ INICIAR SIMULACIÓN

1. Simple (sin averías)
2. Compleja (con mecánicos)
3. Muy Compleja (mecánicos + apagones)
0. Volver al menú principal

Tipo de simulación: 1

[NOTIFICACIÓN DASHBOARD]

Mensaje: Iniciando simulación con 1 vehículos...

--- T=1 s ---

--- T=2 s ---

--- T=3 s ---

[NOTIFICACIÓN DASHBOARD]

Mensaje: [Turismo #1] CHASIS -> MOTOR | Motor=Gasolina (1000.0cc, 300CV)

--- T=4 s ---

--- T=5 s ---

--- T=6 s ---

[NOTIFICACIÓN DASHBOARD]

Mensaje: [Turismo #1] MOTOR -> TAPICERIA | Tapicería=Tela (1)

Practica de Programación Orientada a Objeto – Curs 2025-2026

MENÚ LISTADOS Y ESTADÍSTICAS

1. Operarios ordenados (nombre, apellidos)
2. Operarios por productividad (mínimo de montajes)
3. Vehículos terminados
4. Vehículos terminados por tipo de motor
5. Vehículos terminados por tipo de tapicería
6. Vehículos terminados ordenados por tipo
7. Configuraciones más ensambladas
8. Consultar historial por fecha
0. Volver al menú principal

Opción: **8**

Fecha (dd/MM/yyyy) – vacío = hoy:

[NOTIFICACIÓN DASHBOARD]

Mensaje: Historial de operaciones para 17/05/2026:

```

- 15:58:21 | Motor | Añadido al almacén
- 15:59:27 | Tapiceria | Añadida al almacén
- 15:59:30 | Rueda | Añadida al almacén
- 15:59:30 | Rueda | Añadida al almacén
- 15:59:30 | Rueda | Añadida al almacén
- 15:59:30 | Rueda | Añadida al almacén
- 16:01:01 | Turismo #1 | CHASIS -> MOTOR | Motor=Gasolina (1000.0cc, 300CV)
- 16:01:04 | Turismo #1 | MOTOR -> TAPICERIA | Tapicería=Tela (1)
- 16:01:07 | Turismo #1 | TAPICERIA -> RUEDAS | Ruedas=Normal (1mm)
- 16:01:10 | Turismo | Vehículo terminado
- 16:01:10 | Turismo #1 | RUEDAS -> TERMINADO
  
```

7. Anexo Codigo

7.1 com.practica.vehiculo

Vehiculo.java

```

package com.practica.vehiculo;

import com.practica.tapiceria.*;

import java.util.Arrays;

import com.practica.motor.*;
import com.practica.rueda.*;

/**
 * Clase base abstracta que representa un coche genérico fabricado en la factoría.
 * <p>
 * Encapsula las características comunes a todos los tipos de vehículos del catálogo
 * (Biplaza Deportivo, Turismo y Furgoneta): color, número de plazas, peso máximo
 * autorizado, tara y los componentes principales (motor, tapicería y ruedas).
 * Además, mantiene el estado actual del proceso de montaje y la información de
 * averías para que el planificador y los mecánicos puedan gestionarlas.
 * @author Shao Capilla Sanz
  
```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

*/
public abstract class Coche {

    /** Color exterior del vehículo. */
    private String color;
    /** Número de plazas del vehículo. */
    private int plazas;
    /** Peso máximo autorizado (MMA) del vehículo en kg. */
    private double pesoAutorizado;
    /** Tara (peso en vacío) del vehículo en kg. */
    private double taraVehiculo;

    /** Tapicería instalada en el interior del vehículo. */
    private Tapiceria tapiceria;
    /** Motor montado en el vehículo. */
    private Motor motor;
    /** Conjunto de cuatro ruedas instaladas en el vehículo. */
    private Rueda[] rueda = new Rueda[4];

    /** Estado actual del proceso de montaje del vehículo. */
    private EstadoMontaje estado;

    /** Indica si el vehículo está averiado y esperando reparación. */
    private boolean averiado = false;
    /** Tiempo (en segundos de simulación) que resta para finalizar la reparación. */
    private int tiempoReparacion = 0;

    /**
     * Crea un coche con todos sus atributos principales y componentes asignados.
     */
    * @param color      color exterior del vehículo
    * @param plazas      número de plazas
    * @param pesoAutorizado peso máximo autorizado (MMA) en kg
    * @param taraVehiculo tara (peso en vacío) en kg
    * @param tapiceria    tapicería instalada
    * @param motor        motor instalado
    * @param rueda        array con las cuatro ruedas del vehículo
    */
    public Coche(String color, int plazas, double pesoAutorizado, double taraVehiculo,
Tapiceria tapiceria, Motor motor, Rueda[] rueda) {
        this.color = color;
        this.plazas = plazas;
        this.pesoAutorizado = pesoAutorizado;
        this.taraVehiculo = taraVehiculo;
        this.tapiceria = tapiceria;
        this.motor = motor;
        this.rueda = rueda;
    }
}

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```
/**
 * @return color exterior del vehículo
 */
public String getColor() {
    return color;
}

/**
 * Establece el color exterior del vehículo.
 *
 * @param color nuevo color
 */
public void setColor(String color) {
    this.color = color;
}

/**
 * @return número de plazas del vehículo
 */
public int getPlazas() {
    return plazas;
}

/**
 * Establece el número de plazas del vehículo.
 *
 * @param plazas número de plazas
 */
public void setPlazas(int plazas) {
    this.plazas = plazas;
}

/**
 * @return peso máximo autorizado (MMA) en kg
 */
public double getPesoAutorizado() {
    return pesoAutorizado;
}

/**
 * Establece el peso máximo autorizado del vehículo.
 *
 * @param pesoAutorizado peso máximo en kg
 */
public void setPesoAutorizado(double pesoAutorizado) {
    this.pesoAutorizado = pesoAutorizado;
}
```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```
/**
 * @return tara (peso en vacío) del vehículo en kg
 */
public double getTaraVehiculo() {
    return taraVehiculo;
}

/**
 * Establece la tara del vehículo.
 *
 * @param taraVehiculo tara en kg
 */
public void setTaraVehiculo(double taraVehiculo) {
    this.taraVehiculo = taraVehiculo;
}

/**
 * @return tapicería instalada en el vehículo
 */
public Tapiceria getTapiceria() {
    return tapiceria;
}

/**
 * Asigna una tapicería al vehículo.
 *
 * @param tapiceria tapicería a instalar
 */
public void setTapiceria(Tapiceria tapiceria) {
    this.tapiceria = tapiceria;
}

/**
 * @return array de ruedas instaladas en el vehículo
 */
public Rueda[] getRueda() {
    return rueda;
}

/**
 * Asigna el conjunto de ruedas al vehículo.
 *
 * @param rueda array de ruedas a instalar
 */
public void setRueda(Rueda[] rueda) {
    this.rueda = rueda;
}
```


Practica de Programación Orientada a Objeto – Curs 2025-2026

```
/**
 * @return motor instalado en el vehículo
 */
public Motor getMotor() {
    return motor;
}

/**
 * Asigna un motor al vehículo.
 *
 * @param motor motor a instalar
 */
public void setMotor(Motor motor) {
    this.motor = motor;
}

/**
 * @return estado actual del proceso de montaje
 * @see EstadoMontaje
 */
public EstadoMontaje getEstadoMontaje() {
    return estado;
}

/**
 * Establece el estado del proceso de montaje del vehículo.
 *
 * @param estado nuevo estado de montaje
 */
public void setEstadoMontaje(EstadoMontaje estado) {
    this.estado = estado;
}

/**
 * @return {@code true} si el coche está averiado y necesita reparación
 */
public boolean isAveriado() {
    return averiado;
}

/**
 * Marca o limpia el estado de avería del vehículo.
 *
 * @param averiado {@code true} si está averiado
 */
public void setAveriado(boolean averiado) {
    this.averiado = averiado;
}
```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

    }

    /**
     * @return tiempo de reparación restante (en segundos de simulación)
     */
    public int getTiempoReparacion() {
        return tiempoReparacion;
    }

    /**
     * Establece el tiempo de reparación restante.
     *
     * @param tiempoReparacion tiempo en segundos
     */
    public void setTiempoReparacion(int tiempoReparacion) {
        this.tiempoReparacion = tiempoReparacion;
    }

    /**
     * Devuelve la etiqueta textual con el tipo concreto de coche.
     * Cada subclase devuelve su propia denominación.
     *
     * @return tipo del coche (p.ej. "Turismo", "Furgoneta", "Biplaza Deportivo")
     */
    public abstract String tipoCoche();

    /**
     * Representación textual del coche incluyendo todos sus atributos y componentes.
     *
     * @return cadena descriptiva del estado del coche
     */
    @Override
    public String toString() {
        return "Coche [color=" + color + ", plazas=" + plazas + ", pesoAutorizado=" +
            pesoAutorizado + ", taraVehiculo=" + taraVehiculo + ", tapiceria=" + tapiceria + ", motor=" +
            motor + ", rueda=" + Arrays.toString(rueda) + "]\n";
    }
}

```

BiplazaDeportivo.java

```

package com.practica.vehiculo;

import com.practica.motor.Motor;
import com.practica.rueda.Rueda;
import com.practica.tapiceria.Tapiceria;

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

/**
 * Vehículo de tipo Biplaza Deportivo.
 * <p>
 * Subclase concreta de {@link Coche} que representa los coches deportivos de
 * dos plazas del catálogo de la fábrica. Hereda toda la funcionalidad de la
 * clase base y únicamente sobrescribe la identificación de tipo. *
 * @author Shao Capilla Sanz
 */
public class BiplazaDeportivo extends Coche {

    /**
     * Crea un Biplaza Deportivo con todos sus atributos y componentes.
     *
     * @param color      color exterior del vehículo
     * @param plazas     número de plazas (típicamente 2)
     * @param pesoAutorizado peso máximo autorizado en kg
     * @param taraVehiculo tara del vehículo en kg
     * @param tapiceria   tapicería instalada
     * @param motor       motor instalado
     * @param rueda       conjunto de ruedas instaladas
     */
    public BiplazaDeportivo(String color, int plazas, double pesoAutorizado, double
    taraVehiculo, Tapiceria tapiceria, Motor motor, Rueda[] rueda) {
        super(color, plazas, pesoAutorizado, taraVehiculo, tapiceria, motor, rueda);
    }

    /**
     * {@inheritDoc}
     *
     * @return la cadena "Biplaza Deportivo"
     */
    @Override
    public String tipoCoche() {
        return "Biplaza Deportivo";
    }

    /**
     * @return representación textual del coche incluyendo su tipo
     */
    @Override
    public String toString() {
        return super.toString() + " Tipo: " + tipoCoche();
    }
}

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

Turimos.java

```
package com.practica.vehiculo;

import com.practica.motor.Motor;
import com.practica.rueda.Rueda;
import com.practica.tapiceria.Tapiceria;

/**
 * Vehículo de tipo Turismo.
 * <p>
 * Subclase concreta de {@link Coche} que representa los turismos del catálogo
 * de la fábrica. Constituye uno de los tres tipos de vehículos ensamblados en
 * las cadenas de montaje.
 * @author Shao Capilla Sanz
 */
public class Turismo extends Coche {

    /**
     * Crea un Turismo con todos sus atributos y componentes.
     *
     * @param color      color exterior del vehículo
     * @param plazas      número de plazas (típicamente 5)
     * @param pesoAutorizado peso máximo autorizado en kg
     * @param taraVehiculo tara del vehículo en kg
     * @param tapiceria    tapicería instalada
     * @param motor        motor instalado
     * @param rueda        conjunto de ruedas instaladas
     */
    public Turismo(String color, int plazas, double pesoAutorizado, double taraVehiculo,
Tapiceria tapiceria, Motor motor, Rueda[] rueda) {
        super(color, plazas, pesoAutorizado, taraVehiculo, tapiceria, motor, rueda);
    }

    /**
     * {@inheritDoc}
     *
     * @return la cadena "Turismo"
     */
    @Override
    public String tipoCoche() {
        return "Turismo";
    }

    /**
     * @return representación textual del coche incluyendo su tipo
     */
    @Override
    public String toString() {
```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

        return super.toString() + " Tipo: " + tipoCoche();
    }
}

```

Furgoneta.java

```

package com.practica.vehiculo;

import com.practica.motor.Motor;
import com.practica.rueda.Rueda;
import com.practica.tapiceria.Tapiceria;

/**
 * Vehículo de tipo Furgoneta.
 * <p>
 * Subclase concreta de {@link Coche} que representa las furgonetas del catálogo
 * de la fábrica. Es uno de los tres tipos de vehículos ensamblados en las
 * cadenas de montaje de la factoría. *
 * @author Shao Capilla Sanz
 */
public class Furgoneta extends Coche {

    /**
     * Crea una Furgoneta con todos sus atributos y componentes.
     *
     * @param color      color exterior del vehículo
     * @param plazas      número de plazas
     * @param pesoAutorizado peso máximo autorizado en kg
     * @param taraVehiculo tara del vehículo en kg
     * @param tapiceria   tapicería instalada
     * @param motor        motor instalado
     * @param rueda        conjunto de ruedas instaladas
     */
    public Furgoneta(String color, int plazas, double pesoAutorizado, double taraVehiculo,
        Tapiceria tapiceria, Motor motor, Rueda[] rueda) {
        super(color, plazas, pesoAutorizado, taraVehiculo, tapiceria, motor, rueda);
    }

    /**
     * {@inheritDoc}
     *
     * @return la cadena "Furgoneta"
     */
    @Override
    public String tipoCoche() {
        return "Furgoneta";
    }
}

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

/**
 * @return representación textual del coche incluyendo su tipo
 */
@Override
public String toString() {
    return super.toString() + " Tipo: " + tipoCoche();
}
}

```

Estado.java

```

package com.practica.vehiculo;

/**
 * Estados por los que pasa un vehículo durante el proceso de ensamblaje en la
 * cadena de montaje.
 * <p>
 * El orden de los valores refleja la secuencia natural del montaje: primero se
 * coloca el chasis, después el motor, a continuación la tapicería y finalmente
 * las ruedas, quedando el vehículo en estado {@link #TERMINADO} una vez
 * completado. *
 * @author Shao Capilla Sanz
 */
public enum EstadoMontaje {
    /** Fase de montaje del chasis. */
    CHASIS,
    /** Fase de instalación del motor. */
    MOTOR,
    /** Fase de instalación de la tapicería. */
    TAPICERIA,
    /** Fase de instalación de las ruedas. */
    RUEDAS,
    /** Vehículo completamente ensamblado. */
    TERMINADO
}

```

7.2 com.practica.tapiceria

Tapiceria.java

```

package com.practica.tapiceria;

/**
 * Clase base abstracta que representa la tapicería instalable en el interior

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

* de los vehículos de la fábrica.
* <p>
* Define las características comunes a todos los tipos de tapicería (tela,
* cuero y alcántara): color y metros cuadrados de tela. Las subclases
* concretan el tipo específico mediante {@link #tipoTapiceria()}. *
* @author Shao Capilla Sanz
*/
public abstract class Tapiceria {

    /** Color de la tapicería. */
    private String color;
    /** Metros cuadrados de tela empleados. */
    private double metrosCuadrados;

    /**
     * Constructor por defecto.
     */
    public Tapiceria() {

    }

    /**
     * Crea una tapicería con sus características.
     *
     * @param color color de la tapicería
     * @param metrosCuadrados metros cuadrados de tela
     */
    public Tapiceria(String color, double metrosCuadrados) {
        this.color = color;
        this.metrosCuadrados = metrosCuadrados;
    }

    /**
     * @return color de la tapicería
     */
    public String getColor() {
        return color;
    }

    /**
     * Establece el color de la tapicería.
     *
     * @param color nuevo color
     */
    public void setColor(String color) {
        this.color = color;
    }
}

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

/**
 * @return metros cuadrados de tela de la tapicería
 */
public double getMetrosCuadrados() {
    return metrosCuadrados;
}

/**
 * Establece los metros cuadrados de tela.
 *
 * @param metrosCuadrados metros cuadrados
 */
public void setMetrosCuadrados(double metrosCuadrados) {
    this.metrosCuadrados = metrosCuadrados;
}

/**
 * Devuelve la etiqueta textual con el tipo concreto de tapicería.
 *
 * @return tipo de tapicería (p.ej. "Tela", "Cuero", "Alcantara")
 */
public abstract String tipoTapiceria();

/**
 * @return representación textual de la tapicería con sus características
 */
@Override
public String toString() {
    return "Tapiceria [color=" + color + ", metrosCuadrados=" + metrosCuadrados + "];"
}
}

```

Alcantara.java

```

package com.practica.tapiceria;

/**
 * Tapicería de tipo Alcántara.
 *
 * @author Shao Capilla Sanz
 */
public class Alcantara extends Tapiceria {

    /**
     * Crea una tapicería de alcántara.
     *
     * @param color color de la tapicería

```


Practica de Programación Orientada a Objeto – Curs 2025-2026

```

    * @param metrosCuadrados metros cuadrados de tela
    */
    public Alcantara(String color, double metrosCuadrados) {
        super(color, metrosCuadrados);
    }

    /**
     * {@inheritDoc}
     *
     * @return la cadena "Alcantara"
     */
    @Override
    public String tipoTapiceria() {
        return "Alcantara";
    }

    /**
     * @return representación textual incluyendo el tipo de tapicería
     */
    @Override
    public String toString() {
        return super.toString() + " Tipo: " + tipoTapiceria();
    }
}

```

Tela.java

```

package com.practica.tapiceria;

/**
 * Tapicería de tipo Tela.
 *
 * @author Shao Capilla Sanz
 */
public class Tela extends Tapiceria {

    /**
     * Crea una tapicería de tela.
     *
     * @param color color de la tapicería
     * @param metrosCuadrados metros cuadrados de tela
     */
    public Tela(String color, double metrosCuadrados) {
        super(color, metrosCuadrados);
    }

    /**

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

    * {@inheritDoc}
    *
    * @return la cadena "Tela"
    */
    @Override
    public String tipoTapiceria() {
        return "Tela";
    }

    /**
     * @return representación textual incluyendo el tipo de tapicería
     */
    @Override
    public String toString() {
        return super.toString() + " Tipo: " + tipoTapiceria();
    }
}

```

Cuero.java

```

package com.practica.tapiceria;

/**
 * Tapicería de tipo Cuero.
 *
 * @author Shao Capilla Sanz
 */
public class Cuero extends Tapiceria {

    /**
     * Crea una tapicería de cuero.
     *
     * @param color      color de la tapicería
     * @param metrosCuadrados metros cuadrados de tela
     */
    public Cuero(String color, double metrosCuadrados) {
        super(color, metrosCuadrados);
    }

    /**
     * {@inheritDoc}
     *
     * @return la cadena "Cuero"
     */
    @Override
    public String tipoTapiceria() {
        return "Cuero";
    }
}

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

    }

    /**
     * @return representación textual incluyendo el tipo de tapicería
     */
    @Override
    public String toString() {
        return super.toString() + " Tipo: " + tipoTapiceria();
    }
}

```

7.3 com.practica.rueda

Rueda.java

```

package com.practica.rueda;

/**
 * Clase base abstracta que representa una rueda montable en los vehículos
 * de la fábrica.
 * <p>
 * Define las características técnicas comunes a todos los tipos de rueda
 * (normal, deportiva y todoterreno): ancho en mm, diámetro de llanta en
 * pulgadas, índice de carga en kg y código de velocidad. Este último
 * indica la velocidad máxima permitida que el neumático puede soportar
 * con seguridad. *
 * @author Shao Capilla Sanz
 */
public abstract class Rueda {

    /** Ancho del neumático en milímetros. */
    private int ancho;
    /** Diámetro de la llanta en pulgadas. */
    private int pulgadasLlanta;
    /** Índice de carga del neumático en kg. */
    private int indiceCarga;
    /** Código de velocidad (km/h) que indica la velocidad máxima soportada. */
    private int codigoVelocidad;

    /**
     * Constructor por defecto.
     */
    public Rueda() {

    }
}

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

/**
 * Crea una rueda con todas sus características técnicas.
 *
 * @param ancho      ancho en mm
 * @param pulgadasLlanta  diámetro de llanta en pulgadas
 * @param indiceCarga  índice de carga en kg
 * @param codigoVelocidad código de velocidad en km/h
 */
public Rueda(int ancho, int pulgadasLlanta, int indiceCarga, int codigoVelocidad) {
    this.ancho = ancho;
    this.pulgadasLlanta = pulgadasLlanta;
    this.indiceCarga = indiceCarga;
    this.codigoVelocidad = codigoVelocidad;
}

/**
 * @return ancho del neumático en mm
 */
public int getAncho() {
    return ancho;
}

/**
 * Establece el ancho del neumático.
 *
 * @param ancho ancho en mm
 */
public void setAncho(int ancho) {
    this.ancho = ancho;
}

/**
 * @return diámetro de llanta en pulgadas
 */
public int getPulgadasLlanta() {
    return pulgadasLlanta;
}

/**
 * Establece el diámetro de la llanta.
 *
 * @param pulgadasLlanta diámetro en pulgadas
 */
public void setPulgadasLlanta(int pulgadasLlanta) {
    this.pulgadasLlanta = pulgadasLlanta;
}

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

/**
 * @return índice de carga del neumático en kg
 */
public int getIndiceCarga() {
    return indiceCarga;
}

/**
 * Establece el índice de carga del neumático.
 *
 * @param indiceCarga índice de carga en kg
 */
public void setIndiceCarga(int indiceCarga) {
    this.indiceCarga = indiceCarga;
}

/**
 * @return código de velocidad del neumático (km/h)
 */
public int getVelocidad() {
    return codigoVelocidad;
}

/**
 * Establece el código de velocidad del neumático.
 *
 * @param codigoVelocidad código de velocidad en km/h
 */
public void setVelocidad(int codigoVelocidad) {
    this.codigoVelocidad = codigoVelocidad;
}

/**
 * Devuelve la etiqueta textual con el tipo concreto de rueda.
 *
 * @return tipo de rueda (p.ej. "Normal", "Deportivo", "Todoterreno")
 */
public abstract String tipoRueda();

/**
 * @return representación textual de la rueda con sus características
 */
@Override
public String toString() {
    return "Rueda [ancho=" + ancho + ", pulgadasLlanta=" + pulgadasLlanta + ", indiceCarga=" + indiceCarga + ", codigoVelocidad=" + codigoVelocidad + "];"
}
}

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

Todoterreno.java

```
package com.practica.rueda;

/**
 * Rueda de tipo Todoterreno, apta para terrenos irregulares.
 *
 * @author Shao Capilla Sanz
 */
public class Todoterreno extends Rueda {

    /**
     * Crea una rueda todoterreno con sus características técnicas.
     *
     * @param ancho ancho en mm
     * @param pulgadasLlanta diámetro de llanta en pulgadas
     * @param indiceCarga índice de carga en kg
     * @param codigoVelocidad código de velocidad en km/h
     */
    public Todoterreno(int ancho, int pulgadasLlanta, int indiceCarga, int codigoVelocidad) {
        super(ancho, pulgadasLlanta, indiceCarga, codigoVelocidad);
    }

    /**
     * {@inheritDoc}
     *
     * @return la cadena "Todoterreno"
     */
    @Override
    public String tipoRueda() {
        return "Todoterreno";
    }

    /**
     * @return representación textual de la rueda incluyendo su tipo
     */
    @Override
    public String toString() {
        return super.toString() + " Tipo: " + tipoRueda() + "];";
    }
}
```

Practica de Programación Orientada a Objeto – Curs 2025-2026

Normal.java

```
package com.practica.rueda;

/**
 * Rueda de tipo Normal para uso general.
 *
 * @author Shao Capilla Sanz
 */
public class Normal extends Rueda {

    /**
     * Crea una rueda normal con sus características técnicas.
     *
     * @param ancho      ancho en mm
     * @param pulgadasLlanta  diámetro de llanta en pulgadas
     * @param indiceCarga    índice de carga en kg
     * @param codigoVelocidad código de velocidad en km/h
     */
    public Normal(int ancho, int pulgadasLlanta, int indiceCarga, int codigoVelocidad) {
        super(ancho, pulgadasLlanta, indiceCarga, codigoVelocidad);
    }

    /**
     * @inheritDoc
     *
     * @return la cadena "Normal"
     */
    @Override
    public String tipoRueda() {
        return "Normal";
    }

    /**
     * @return representación textual de la rueda incluyendo su tipo
     */
    @Override
    public String toString() {
        return super.toString() + " Tipo: " + tipoRueda() + " ";
    }
}
```

Deportivo.java

```
package com.practica.rueda;

/**
 * Rueda de tipo Deportivo, orientada a vehículos de altas prestaciones.
```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```
*
* @author Shao Capilla Sanz
*/
public class Deportivo extends Rueda {

    /**
     * Crea una rueda deportiva con sus características técnicas.
     *
     * @param ancho      ancho en mm
     * @param pulgadasLlanta  diámetro de llanta en pulgadas
     * @param indiceCarga    índice de carga en kg
     * @param codigoVelocidad código de velocidad en km/h
     */
    public Deportivo(int ancho, int pulgadasLlanta, int indiceCarga, int codigoVelocidad) {
        super(ancho, pulgadasLlanta, indiceCarga, codigoVelocidad);
    }

    /**
     * {@inheritDoc}
     *
     * @return la cadena "Deportivo"
     */
    @Override
    public String tipoRueda() {
        return "Deportivo";
    }

    /**
     * @return representación textual de la rueda incluyendo su tipo
     */
    @Override
    public String toString() {
        return super.toString() + " Tipo: " + tipoRueda() + "];
    }
}
```


Practica de Programación Orientada a Objeto – Curs 2025-2026

7.4 com.practica.motor

Motor.java

```
package com.practica.motor;

/**
 * Clase base abstracta que representa un motor de los que se pueden montar
 * en los vehículos de la fábrica.
 * <p>
 * Define las características técnicas comunes a todos los tipos de motor
 * (eléctrico, gasolina e híbrido): cilindrada, potencia y número de cilindros.
 * Las subclases concretan el tipo específico de motor mediante el método
 * {@link #tipoMotor()}.
 * @author Shao Capilla Sanz
 */
public abstract class Motor {

    /** Cilindrada del motor (cc). */
    private double cilindrada;
    /** Potencia del motor en caballos de vapor (CV). */
    private int potencia;
    /** Número de cilindros del motor. */
    private int numeroCilindros;

    /**
     * Constructor por defecto.
     */
    public Motor() {

    }

    /**
     * Crea un motor con todas sus características técnicas.
     *
     * @param cilindrada cilindrada en cc
     * @param potencia potencia en CV
     * @param numeroCilindros número de cilindros
     */
    public Motor(double cilindrada, int potencia, int numeroCilindros) {
        this.cilindrada = cilindrada;
        this.potencia = potencia;
        this.numeroCilindros = numeroCilindros;
    }

    /**
     * @return cilindrada del motor en cc
     */
}
```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```
public double getCilindrada() {
    return cilindrada;
}

/**
 * Establece la cilindrada del motor.
 *
 * @param cilindrada cilindrada en cc
 */
public void setCilindrada(double cilindrada) {
    this.cilindrada = cilindrada;
}

/**
 * @return potencia del motor en CV
 */
public int getPotencia() {
    return potencia;
}

/**
 * Establece la potencia del motor.
 *
 * @param potencia potencia en CV
 */
public void setPotencia(int potencia) {
    this.potencia = potencia;
}

/**
 * @return número de cilindros del motor
 */
public int getNumeroCilindros() {
    return numeroCilindros;
}

/**
 * Establece el número de cilindros del motor.
 *
 * @param numeroCilindros número de cilindros
 */
public void setNumeroCilindros(int numeroCilindros) {
    this.numeroCilindros = numeroCilindros;
}

/**
 * Devuelve la etiqueta textual con el tipo concreto de motor.
 */
```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

    * @return tipo del motor (p.ej. "Gasolina", "Electrico", "Hibrido")
    */
    public abstract String tipoMotor();

    /**
     * @return representación textual del motor con sus características técnicas
     */
    @Override
    public String toString() {
        return "Motor [cilindrada=" + cilindrada + ", potencia=" + potencia + ",
        numeroCilindros=" + numeroCilindros + "]";
    }
}

```

Hibrido.java

```

package com.practica.motor;

/**
 * Motor híbrido montable en los vehículos de la fábrica.
 *
 * @author Shao Capilla Sanz
 */
public class Hibrido extends Motor {

    /**
     * Crea un motor híbrido con sus características técnicas.
     *
     * @param cilindrada    cilindrada en cc
     * @param potencia      potencia en CV
     * @param numeroCilindros número de cilindros
     */
    public Hibrido(double cilindrada, int potencia, int numeroCilindros) {
        super(cilindrada, potencia, numeroCilindros);
    }

    /**
     * {@inheritDoc}
     *
     * @return la cadena "Hibrido"
     */
    @Override
    public String tipoMotor() {
        return "Hibrido";
    }

    /**

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

    * @return representación textual del motor incluyendo su tipo
    */
    @Override
    public String toString() {
        return super.toString() + " Tipo: " + tipoMotor();
    }
}

```

Gasolina.java

```

package com.practica.motor;

/**
 * Motor de gasolina montable en los vehículos de la fábrica.
 *
 * @author Shao Capilla Sanz
 */
public class Gasolina extends Motor {

    /**
     * Crea un motor de gasolina con sus características técnicas.
     *
     * @param cilindrada    cilindrada en cc
     * @param potencia      potencia en CV
     * @param numeroCilindros número de cilindros
     */
    public Gasolina(double cilindrada, int potencia, int numeroCilindros) {
        super(cilindrada, potencia, numeroCilindros);
    }

    /**
     * {@inheritDoc}
     *
     * @return la cadena "Gasolina"
     */
    @Override
    public String tipoMotor() {
        return "Gasolina";
    }

    /**
     * @return representación textual del motor incluyendo su tipo
     */
    @Override
    public String toString() {
        return super.toString() + " Tipo: " + tipoMotor();
    }
}

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```
}
}
```

Electrico.java

```
package com.practica.motor;

/**
 * Motor eléctrico montable en los vehículos de la fábrica.
 *
 * @author Shao Capilla Sanz
 */
public class Electrico extends Motor {

    /**
     * Crea un motor eléctrico con sus características técnicas.
     *
     * @param cilindrada    cilindrada en cc
     * @param potencia      potencia en CV
     * @param numeroCilindros número de cilindros
     */
    public Electrico(double cilindrada, int potencia, int numeroCilindros) {
        super(cilindrada, potencia, numeroCilindros);
    }

    /**
     * {@inheritDoc}
     *
     * @return la cadena "Electrico"
     */
    @Override
    public String tipoMotor() {
        return "Electrico";
    }

    /**
     * @return representación textual del motor incluyendo su tipo
     */
    @Override
    public String toString() {
        return super.toString() + " Tipo: " + tipoMotor();
    }
}
```

Practica de Programación Orientada a Objeto – Curs 2025-2026

7.5 com.practica.personal

Trabajador.java

```
package com.practica.personal;

import java.time.LocalDate;

/**
 * Clase base abstracta que representa a un trabajador de la fábrica.
 * <p>
 * Encapsula los datos personales y laborales comunes a todos los perfiles
 * (operario, mecánico de cinta, gestor de planta y administrador del
 * sistema): nombre, apellidos, DNI, dirección, número de seguridad social,
 * puesto de trabajo, salario y fecha de ingreso.
 * @author Shao Capilla Sanz
 */
public abstract class Trabajador {

    /** Datos personales y laborales del trabajador. */
    private String nombre, apellidos, dni, direccion, numSegSocial, puestoTrabajo;
    /** Fecha de ingreso en la empresa. */
    private LocalDate fechaIngreso;
    /** Salario bruto del trabajador. */
    private double salario;

    /**
     * Crea un trabajador con todos sus datos personales y laborales.
     *
     * @param nombre    nombre del trabajador
     * @param apellidos apellidos del trabajador
     * @param dni       DNI del trabajador
     * @param direccion dirección postal
     * @param numSegSocial número de la Seguridad Social
     * @param puestoTrabajo puesto que desempeña en la fábrica
     * @param salario    salario bruto
     * @param fechaIngreso fecha de ingreso en la empresa
     */
    public Trabajador(String nombre, String apellidos, String dni, String direccion, String
numSegSocial, String puestoTrabajo, double salario, LocalDate fechaIngreso) {
        this.nombre = nombre;
        this.apellidos = apellidos;
        this.dni = dni;
        this.direccion = direccion;
        this.numSegSocial = numSegSocial;
        this.puestoTrabajo = puestoTrabajo;
        this.salario = salario;
        this.fechaIngreso = fechaIngreso;
    }
}
```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```
}

/**
 * @return nombre del trabajador
 */
public String getNombre() {
    return nombre;
}

/**
 * Establece el nombre del trabajador.
 *
 * @param nombre nuevo nombre
 */
public void setNombre(String nombre) {
    this.nombre = nombre;
}

/**
 * @return apellidos del trabajador
 */
public String getApellidos() {
    return apellidos;
}

/**
 * Establece los apellidos del trabajador.
 *
 * @param apellidos nuevos apellidos
 */
public void setApellidos(String apellidos) {
    this.apellidos = apellidos;
}

/**
 * @return DNI del trabajador
 */
public String getDni() {
    return dni;
}

/**
 * Establece el DNI del trabajador.
 *
 * @param dni nuevo DNI
 */
public void setDni(String dni) {
    this.dni = dni;
}
```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```
}

/**
 * @return dirección postal del trabajador
 */
public String getDireccion() {
    return direccion;
}

/**
 * Establece la dirección postal del trabajador.
 *
 * @param direccion nueva dirección
 */
public void setDireccion(String direccion) {
    this.direccion = direccion;
}

/**
 * @return número de Seguridad Social del trabajador
 */
public String getNumSeguridadSocial() {
    return numSegSocial;
}

/**
 * Establece el número de Seguridad Social del trabajador.
 *
 * @param numSegSocial nuevo número
 */
public void setNumSeguridadSocial(String numSegSocial) {
    this.numSegSocial = numSegSocial;
}

/**
 * @return puesto de trabajo que desempeña
 */
public String getPuestoTrabajo() {
    return puestoTrabajo;
}

/**
 * Establece el puesto de trabajo del trabajador.
 *
 * @param puestoTrabajo nuevo puesto
 */
public void setPuesto(String puestoTrabajo) {
    this.puestoTrabajo = puestoTrabajo;
}
```


Practica de Programación Orientada a Objeto – Curs 2025-2026

```

    }

    /**
     * @return salario bruto del trabajador
     */
    public double getSalario() {
        return salario;
    }

    /**
     * Establece el salario bruto del trabajador.
     *
     * @param salario nuevo salario
     */
    public void setSalario(double salario) {
        this.salario = salario;
    }

    /**
     * @return fecha de ingreso en la empresa
     */
    public LocalDate getfechaIngreso() {
        return fechaIngreso;
    }

    /**
     * Establece la fecha de ingreso del trabajador.
     *
     * @param fechaIngreso nueva fecha de ingreso
     */
    public void setfechaIngreso(LocalDate fechaIngreso) {
        this.fechaIngreso = fechaIngreso;
    }
}

```

AdministradorSistema.java

```

package com.practica.personal;

import java.time.LocalDate;

import com.practica.planificador.Planificador;

/**
 * Trabajador con perfil de Administrador del Sistema.
 * <p>
 * Es el responsable de velar por el correcto funcionamiento del software de

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

* la cadena de montaje y del gestor de fábrica. Cuando se produce un evento
* crítico (por ejemplo, un apagón), todos los trabajadores quedan parados
* hasta que el administrador restaura los sistemas: necesita 2 segundos para
* reanudar el sistema de gestión y 3 segundos para reanudar las cadenas de
* montaje. *
* @author Shao Capilla Sanz
*/
public class AdministradorSistema extends Trabajador {

    /** Número de restauraciones del sistema realizadas. */
    private int restauracionesRealizadas;

    /**
     * Crea un Administrador del Sistema con sus datos personales. El puesto
     * queda fijado automáticamente como "Administrador del sistema".
     *
     * @param nombre nombre del administrador
     * @param apellidos apellidos del administrador
     * @param dni DNI del administrador
     * @param direccion dirección postal
     * @param numSegSocial número de la Seguridad Social
     * @param salario salario bruto
     * @param fechaIngreso fecha de ingreso en la empresa
     */
    public AdministradorSistema(String nombre, String apellidos, String dni, String direccion,
    String numSegSocial, double salario, LocalDate fechaIngreso) {
        super(nombre, apellidos, dni, direccion, numSegSocial, "Administrador del sistema",
    salario, fechaIngreso);
        this.restauracionesRealizadas = 0;
    }

    /**
     * Restaura el sistema de gestión de la fábrica si se encuentra bloqueado,
     * notificando a los observadores del planificador y registrando la
     * restauración en el contador del administrador.
     *
     * @param planificador planificador cuyo sistema de gestión se desbloqueará
     */
    public void restaurarSistemaGestion(Planificador planificador) {
        if (planificador.isSistemaGestionBloqueado()) {
            planificador.setSistemaGestionBloqueado(false);
            restauracionesRealizadas++;
            String msg = "El administrador " + getNombre() + " " + getApellidos() + " ha
    restaurado el sistema de gestión de la fábrica.";
            planificador.notifyObservadores(msg);
        }
    }
}

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

/**
 * Reanuda las cadenas de montaje si están detenidas por una caída de luz,
 * notificando a los observadores del planificador.
 *
 * @param planificador planificador cuyas cadenas se reanudarán
 */
public void restaurarCadenasMontaje(Planificador planificador) {
    if (planificador.isCaidaDeLuz()) {
        planificador.setCaidaDeLuz(false);
        String msg = "El administrador " + getNombre() + " " + getApellidos() + " ha
reanudado las cadenas de montaje.";
        planificador.notifyObservadores(msg);
    }
}

/**
 * @return número total de restauraciones del sistema realizadas
 */
public int getRestauracionesRealizadas() {
    return restauracionesRealizadas;
}
}

```

GestorPlanta.java

```

package com.practica.personal;

import java.time.LocalDate;
import java.util.Date;

import com.practica.dashboard.Dashboard;
import com.practica.montaje.CadenaMontaje;
import com.practica.motor.Motor;
import com.practica.rueda.Rueda;
import com.practica.tapiceria.Tapiceria;
import com.practica.vehiculo.*;

/**
 * Trabajador con perfil de Gestor de Planta.
 * <p>
 * Es el encargado de configurar las cadenas de montaje y supervisar su
 * funcionamiento. Configura los componentes que utilizarán las cadenas
 * para ensamblar los vehículos, monitoriza el dashboard para detectar
 * incidencias y llama a los mecánicos cuando se produce una avería.
 *
 * @author Shao Capilla Sanz
 */
public class GestorPlanta extends Trabajador {

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

/** Dashboard al que el gestor notifica confirmaciones e incidencias. */
private Dashboard dashboard;

/**
 * Crea un Gestor de Planta con sus datos personales y el dashboard
 * que utilizará para notificar el estado de las cadenas de montaje.
 * El puesto queda fijado automáticamente como "Gestor de planta".
 */
* @param nombre    nombre del gestor
* @param apellidos  apellidos del gestor
* @param dni        DNI del gestor
* @param direccion  dirección postal
* @param numSegSocial número de la Seguridad Social
* @param salario    salario bruto
* @param fechaIngreso fecha de ingreso en la empresa
* @param dashboard  dashboard al que se enviarán las notificaciones
*/
public GestorPlanta(String nombre, String apellidos, String dni, String direccion, String
numSegSocial, double salario, LocalDate fechaIngreso, Dashboard dashboard) {
    super(nombre, apellidos, dni, direccion, numSegSocial, "Gestor de planta", salario,
fechaIngreso);
    this.dashboard = dashboard;
}

/**
 * Configura un lote de Biplazas Deportivos para su ensamblaje en la
 * cadena indicada, encolando las unidades en estado {@link EstadoMontaje#CHASIS}.
 * Notifica la acción al dashboard.
 */
* @param cadena    cadena de montaje destino
* @param cantidad  número de unidades a encolar
* @param color     color de los vehículos
* @param tara      tara en kg
* @param pesoMax   peso máximo autorizado en kg
* @param motor     motor a montar (puede ser {@code null} si se asignará después)
* @param tapiceria tapicería a instalar (puede ser {@code null})
* @param ruedas    ruedas a instalar (puede ser {@code null})
*/
public void configurarBiplazas(CadenaMontaje cadena, int cantidad, String color,
double tara, double pesoMax, Motor motor, Tapiceria tapiceria, Rueda[] ruedas) {
    dashboard.update("[GESTOR] " + getNombre() + " configura " + cantidad + "
Biplaza(s) en la cadena de montaje.");
    for (int i = 0; i < cantidad; i++) {
        BiplazaDeportivo bd = new BiplazaDeportivo(color, 2, pesoMax, tara, tapiceria,
motor, ruedas);
        bd.setEstadoMontaje(EstadoMontaje.CHASIS);
        cadena.agregarBiplaza(bd);
    }
}

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

    }
}

/**
 * Configura un lote de Turismos para su ensamblaje en la cadena indicada,
 * encolando las unidades en estado {@link EstadoMontaje#CHASIS}.
 * Notifica la acción al dashboard.
 *
 * @param cadena cadena de montaje destino
 * @param cantidad número de unidades a encolar
 * @param color color de los vehículos
 * @param tara tara en kg
 * @param pesoMax peso máximo autorizado en kg
 * @param motor motor a montar
 * @param tapiceria tapicería a instalar
 * @param ruedas ruedas a instalar
 */
public void configurarTurismos(CadenaMontaje cadena, int cantidad, String color,
double tara, double pesoMax, Motor motor, Tapiceria tapiceria, Rueda[] ruedas) {
    dashboard.update("[GESTOR] " + getNombre() + " configura " + cantidad + "
Turismo(s) en la cadena de montaje.");
    for (int i = 0; i < cantidad; i++) {
        Turismo t = new Turismo(color, 5, pesoMax, tara, tapiceria, motor, ruedas);
        t.setEstadoMontaje(EstadoMontaje.CHASIS);
        cadena.agregarTurismo(t);
    }
}

/**
 * Configura un lote de Furgonetas para su ensamblaje en la cadena indicada,
 * encolando las unidades en estado {@link EstadoMontaje#CHASIS}.
 * Notifica la acción al dashboard.
 *
 * @param cadena cadena de montaje destino
 * @param cantidad número de unidades a encolar
 * @param color color de los vehículos
 * @param tara tara en kg
 * @param pesoMax peso máximo autorizado en kg
 * @param motor motor a montar
 * @param tapiceria tapicería a instalar
 * @param ruedas ruedas a instalar
 */
public void configurarFurgonetas(CadenaMontaje cadena, int cantidad, String color,
double tara, double pesoMax, Motor motor, Tapiceria tapiceria, Rueda[] ruedas) {
    dashboard.update("[GESTOR] " + getNombre() + " configura " + cantidad + "
Furgoneta(s) en la cadena de montaje.");
    for (int i = 0; i < cantidad; i++) {
        Furgoneta f = new Furgoneta(color, 3, pesoMax, tara, tapiceria, motor,

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

ruedas);
        f.setEstadoMontaje(EstadoMontaje.CHASIS);
        cadena.agregarFurgoneta(f);
    }
}

/**
 * Notifica al dashboard que el gestor está revisando el estado de las
 * cadenas de montaje en busca de incidencias.
 */
public void consultarDashboard() {
    dashboard.update("[GESTOR] " + getNombre() + " " + getApellidos() + " revisa el
dashboard para detectar incidencias.");
}

/**
 * Llama a un mecánico para que repare un coche averiado y notifica
 * la incidencia al dashboard.
 *
 * @param mecanico    mecánico al que se llama
 * @param cocheAveriado coche que requiere reparación
 */
public void llamarMecanico(Mecanico mecanico, Coche cocheAveriado) {
    mecanico.repararCoche(cocheAveriado);
    dashboard.update("[GESTOR] " + getNombre() + " " + getApellidos() + " llama al
mecánico " + mecanico.getNombre() + " " + mecanico.getApellidos() + " para reparar una
avería.");
}
}

```

Mecanico.java

```

package com.practica.personal;

import java.time.LocalDate;

import com.practica.dashboard.Dashboard;
import com.practica.vehiculo.Coche;

/**
 * Trabajador con perfil de Mecánico de cinta.
 * <p>
 * Los mecánicos se encargan de reparar las averías que se producen en la
 * cadena de montaje. Existen dos perfiles según la experiencia acumulada:
 * <ul>
 * <li><b>Eficiente</b>: ha realizado más de 20 reparaciones y tarda 1
 * segundo de simulación en reparar cualquier problema.</li>

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

* <li><b>Estándar</b>: requiere entre 2 y 5 segundos por reparación.</li>
* </ul>
*
* @author Shao Capilla Sanz
*/
public class Mecanico extends Trabajador {

    /** Número de reparaciones realizadas por el mecánico. */
    private int reparacionesRealizadas;

    /** Dashboard al que el mecánico notifica el resultado de las reparaciones. */
    private Dashboard dashboard;

    /**
     * Crea un mecánico de cinta con sus datos personales y el dashboard
     * al que notificará el resultado de sus reparaciones.
     * El puesto queda fijado automáticamente como "Mecánico de cinta".
     */
    * @param nombre    nombre del mecánico
    * @param apellidos  apellidos del mecánico
    * @param dni        DNI del mecánico
    * @param direccion  dirección postal
    * @param numSegSocial número de la Seguridad Social
    * @param salario    salario bruto
    * @param fechaIngreso fecha de ingreso en la empresa
    * @param dashboard  dashboard al que se enviarán las notificaciones
    */
    public Mecanico(String nombre, String apellidos, String dni, String direccion, String
numSegSocial, double salario, LocalDate fechaIngreso, Dashboard dashboard) {
        super(nombre, apellidos, dni, direccion, numSegSocial, "Mecánico de cinta", salario,
fechaIngreso);
        this.reparacionesRealizadas = 0;
        this.dashboard = dashboard;
    }

    /**
     * Repara la avería de un coche, limpiando su estado de avería e
     * incrementando el contador de reparaciones del mecánico.
     */
    * @param c coche averiado a reparar; si es {@code null} o no está
    * averiado, no se realiza ninguna acción
    */
    public void repararCoche(Coche c) {
        if (c != null && c.isAveriado()) {
            c.setTiempoReparacion(0);
            c.setAveriado(false);
            reparacionesRealizadas++;
            dashboard.update("El mecánico " + getNombre() + " " + getApellidos() + " ha

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

reparado la avería de un coche.");
    }
}

/**
 * Indica si el mecánico es considerado eficiente, es decir, si ha
 * realizado más de 20 reparaciones.
 *
 * @return {@code true} si es eficiente; {@code false} si es estándar
 */
public boolean esEficiente() {
    return reparacionesRealizadas > 20;
}

/**
 * Devuelve el tiempo (en segundos de simulación) que el mecánico tarda
 * en reparar una avería según su perfil.
 *
 * @return 1 si es eficiente; un valor aleatorio entre 2 y 5 si es estándar
 */
public int getTiempoReparacion() {
    if (esEficiente()) {
        return 1;
    } else {
        return 2 + (int) (Math.random() * 4);
    }
}

/**
 * @return número total de reparaciones realizadas
 */
public int getReparacionesRealizadas() {
    return reparacionesRealizadas;
}
}

```

Operario.java

```

package com.practica.personal;

import java.time.LocalDate;

/**
 * Trabajador con perfil de Operario de la cadena de montaje.
 *
 * 

* Los operarios controlan los robots encargados del montaje de cada
 * componente (chasis, motor, tapicería, ruedas). Existen dos perfiles


```


Practica de Programación Orientada a Objeto – Curs 2025-2026

```

* según la experiencia acumulada:
* <ul>
* <li><b>Eficiente</b></li>: ha realizado más de 10 montajes y completa su tarea
*   en 1 segundo de simulación.</li>
* <li><b>Estándar</b></li>: requiere 3 segundos por tarea.</li>
* </ul>
*
* @author Shao Capilla Sanz
*/
public class Operario extends Trabajador {

    /** Número de montajes de componentes realizados por el operario. */
    private int montajesRealizados;

    /**
     * Crea un operario con sus datos personales. El puesto queda fijado
     * automáticamente como "Operario" y el contador de montajes empieza en 0.
     *
     * @param nombre    nombre del operario
     * @param apellidos  apellidos del operario
     * @param dni        DNI del operario
     * @param direccion  dirección postal
     * @param numSegSocial número de la Seguridad Social
     * @param salario    salario bruto
     * @param fechaIngreso fecha de ingreso en la empresa
     */
    public Operario(String nombre, String apellidos, String dni, String direccion, String
numSegSocial, double salario, LocalDate fechaIngreso) {
        super(nombre, apellidos, dni, direccion, numSegSocial, "Operario", salario,
fechaIngreso);
        this.montajesRealizados = 0;
    }

    /**
     * Indica si el operario es considerado eficiente, es decir, si ha
     * realizado más de 10 montajes.
     *
     * @return {@code true} si es eficiente; {@code false} si es estándar
     */
    public boolean esEficiente(){
        return montajesRealizados > 10;
    }

    /**
     * Devuelve el tiempo en segundos que el operario tarda en realizar un
     * montaje según su perfil.
     *
     * @return 1 si es eficiente, 3 si es estándar

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

*/
public int getTiempoMontaje(){
    return esEficiente() ? 1 : 3;
}

/**
 * Incrementa en uno el contador de montajes completados por el operario.
 * Se invoca cuando finaliza con éxito una tarea de ensamblaje.
 */
public void registrarMontajeCompletado() {
    this.montajesRealizados++;
}

/**
 * @return número total de montajes realizados
 */
public int getMontajesRealizados() {
    return montajesRealizados;
}

/**
 * Establece el número de montajes realizados (utilizable para inicializar
 * operarios con experiencia previa).
 *
 * @param montajesRealizados nuevo contador
 */
public void setMontajesRealizados(int montajesRealizados) {
    this.montajesRealizados = montajesRealizados;
}
}

```

7.6 com.practica.dashboard

Dashboard.java

```

package com.practica.dashboard;

/**
 * Cuadro de mandos (dashboard) del sistema de gestión de la fábrica.
 * <p>
 * Permite al gestor de planta consultar el estado en tiempo real de las
 * cadenas de montaje y del almacén. Implementa el patrón Observador
 * registrándose como {@link Observador} sobre las entidades observables
 * (cadena de montaje, almacén, planificador) para recibir notificaciones
 * cada vez que se produce un cambio relevante. * <p>
 * El subsistema concreto de visualización está desacoplado a través de la

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

* interfaz {@link IfaceVisualizarDatos}, lo que facilita cambiar la forma
* de mostrar los datos (consola, gráfica, etc.) sin tocar la lógica. *
* @author Shao Capilla Sanz
*/
public class Dashboard implements Observador {

    /**
     * Sistema concreto encargado de visualizar los datos. */
    private IfaceVisualizarDatos visualizador;

    /**
     * Crea un dashboard que delegará la visualización en el subsistema
     * indicado.
     *
     * @param visualizador subsistema de visualización a utilizar
     */
    public Dashboard(IfaceVisualizarDatos visualizador) {
        this.visualizador = visualizador;
    }

    /**
     * {@inheritDoc}
     * <p>
     * Reenvía el mensaje recibido al subsistema de visualización configurado.
     */
    @Override
    public void update(String message) {
        visualizador.mostrarDatos(message);
    }

    /**
     * Muestra un mensaje de confirmación de una acción realizada por el usuario,
     * delegando la visualización en el subsistema configurado.
     *
     * @param message texto de confirmación a mostrar
     */
    public void mostrarConfirmacion(String message) {
        visualizador.confirmacionDatos(message);
    }

    /**
     * Muestra un mensaje de error al usuario, delegando la visualización
     * en el subsistema configurado.
     *
     * @param message texto del error a mostrar
     */
    public void mostrarError(String message) {
        visualizador.errorDashboard(message);
    }
}

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```
}
```

lfaceVisualizarDatos.java

```
package com.practica.dashboard;

/**
 * Interfaz que define el subsistema de visualización utilizado por el
 * {@link Dashboard}.
 * <p>
 * Su existencia permite desacoplar el dashboard del mecanismo concreto de
 * presentación, de modo que se pueda cambiar fácilmente entre una salida
 * por consola, una interfaz gráfica, registro en fichero, etc., sin
 * modificar el resto del sistema.
 * @author Shao Capilla Sanz
 */
public interface lfaceVisualizarDatos {
    /**
     * Muestra el mensaje indicado a través del medio de visualización
     * concreto.
     *
     * @param mensaje texto a mostrar
     */
    void mostrarDatos(String mensaje);

    /**
     * Muestra un mensaje de confirmación de una acción realizada,
     * diferenciándolo visualmente de las notificaciones de cambio de estado.
     *
     * @param mensaje texto de confirmación a mostrar
     */
    void confirmacionDatos(String mensaje);

    /**
     * Muestra un mensaje de error, diferenciándolo visualmente del resto
     * de mensajes del dashboard.
     *
     * @param mensaje texto del error a mostrar
     */
    void errorDashboard(String mensaje);
}
```

Practica de Programación Orientada a Objeto – Curs 2025-2026

Observable.java

```
package com.practica.dashboard;

/**
 * Interfaz del rol Sujeto (Subject) del patrón Observador.
 * <p>
 * Define el contrato que deben cumplir las entidades del sistema capaces de
 * emitir notificaciones a observadores registrados cuando se produce un
 * cambio en su estado (cadena de montaje, almacén, planificador, etc.). *
 * @author Shao Capilla Sanz
 */
public interface Observable {

    /**
     * Registra un nuevo observador para recibir notificaciones.
     *
     * @param observador observador a añadir
     */
    void addObservador(Observador observador);

    /**
     * Elimina un observador para que deje de recibir notificaciones.
     *
     * @param observador observador a quitar
     */
    void removeObservador(Observador observador);

    /**
     * Notifica un mensaje a todos los observadores registrados.
     *
     * @param message texto del mensaje a difundir
     */
    void notifyObservadores(String message);
}
```

Observador.java

```
package com.practica.dashboard;

/**
 * Interfaz del rol Observador del patrón Observador.
 * <p>
 * Implementada por aquellas entidades que desean ser notificadas cuando un
 * objeto @link Observable} cambia su estado (por ejemplo, el dashboard
 * recibe avisos de la cadena de montaje y del almacén). *
 * @author Shao Capilla Sanz
 */
```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

*/
public interface Observador {

    /**
     * Método invocado por el observable para notificar un cambio.
     *
     * @param message texto descriptivo del evento ocurrido
     */
    void update(String message);
}

```

VisualizarConsola.java

```

package com.practica.dashboard;

/**
 * Implementación de {@link IfaceVisualizarDatos} que muestra los mensajes
 * del dashboard por la consola estándar.
 * <p>
 * Es la implementación por defecto utilizada en la aplicación textual. *
 * @author Shao Capilla Sanz
 */
public class VisualizarConsola implements IfaceVisualizarDatos {
    /**
     * {@inheritDoc}
     * <p>
     * Imprime el mensaje en la consola enmarcado entre separadores para
     * destacarlo del resto de salida del programa. */
    @Override
    public void mostrarDatos(String mensaje) {
        System.out.println("-----");
        System.out.println("[NOTIFICACIÓN DASHBOARD]");
        System.out.println("Mensaje: " + mensaje);
        System.out.println("-----");
    }

    /**
     * {@inheritDoc}
     * <p>
     * Imprime el mensaje en la consola enmarcado entre separadores con
     * la etiqueta @code [CONFIRMACIÓN DASHBOARD]}.
     */
    @Override
    public void confirmacionDatos(String mensaje) {
        System.out.println("-----");
        System.out.println("[CONFIRMACIÓN DASHBOARD]");
    }
}

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

        System.out.println("Mensaje: " + mensaje);
        System.out.println("-----");
    }

    /**
     * {@inheritDoc}
     * <p>
     * Imprime el mensaje en la consola enmarcado entre separadores con
     * la etiqueta {@code [ERROR DASHBOARD]}.
     */
    @Override
    public void errorDashboard(String mensaje) {
        System.out.println("-----");
        System.out.println("[ERROR DASHBOARD]");
        System.out.println("Mensaje: " + mensaje);
        System.out.println("-----");
    }
}

```

7.7 com.practica.fabrica

AlmacenDatos.java

```

package com.practica.fabrica;

import com.practica.vehiculo.*;
import com.practica.rueda.*;
import com.practica.motor.*;
import com.practica.tapiceria.*;
import com.practica.personal.*;
import com.practica.dashboard.Observable;
import com.practica.dashboard.observador;

import java.util.*;

/**
 * Implementación basada en {@link ArrayList} del almacén de datos del sistema.
 * <p>
 * Mantiene en memoria todas las entidades del sistema (vehículos terminados,
 * stock de componentes, plantilla de trabajadores y registro histórico de
 * operaciones de montaje). Además, actúa como sujeto observable del patrón
 * Observador para notificar al dashboard cada vez que cambia el estado del
 * almacén. *
 * @author Shao Capilla Sanz
 */
public class AlmacenDatos implements IfaceAlmacen, Observable {

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

/** Stock de Biplazas Deportivos terminados. */
private ArrayList<BiplazaDeportivo> biplazasDeportivos;
/** Stock de Turismos terminados. */
private ArrayList<Turismo> turismos;
/** Stock de Furgonetas terminadas. */
private ArrayList<Furgoneta> furgonetas;

/** Stock de ruedas disponibles para montaje. */
private ArrayList<Rueda> ruedas;

/** Stock de motores disponibles para montaje. */
private ArrayList<Motor> motores;

/** Stock de tapicerías disponibles para montaje. */
private ArrayList<Tapiceria> tapicerias;

/** Plantilla de administradores del sistema. */
private ArrayList<AdministradorSistema> administradoresSistema;
/** Plantilla de gestores de planta. */
private ArrayList<GestorPlanta> gestoresPlanta;
/** Plantilla de mecánicos de cinta. */
private ArrayList<Mecanico> mecanicos;
/** Plantilla de operarios. */
private ArrayList<Operario> operarios;

/** Observadores registrados para recibir notificaciones del almacén. */
private ArrayList<Observador> observadores;

/** Historial de operaciones de montaje registradas en el sistema. */
private List<RegistroMontaje> historial;

/**
 * Inicializa todas las colecciones del almacén vacías.
 */
public AlmacenDatos() {

    biplazasDeportivos = new ArrayList<>();
    turismos = new ArrayList<>();
    furgonetas = new ArrayList<>();
    ruedas = new ArrayList<>();
    motores = new ArrayList<>();
    tapicerias = new ArrayList<>();
    administradoresSistema = new ArrayList<>();
    gestoresPlanta = new ArrayList<>();
    mecanicos = new ArrayList<>();
    operarios = new ArrayList<>();
    observadores = new ArrayList<>();

```


Practica de Programación Orientada a Objeto – Curs 2025-2026

```

        historial = new ArrayList<>();
    }

    /**
     * {@inheritDoc}
     * <p>Registra la operación en el historial y notifica al dashboard.</p>
     */
    @Override
    public void agregarBiplazaDeportivo(BiplazaDeportivo biplazaDeportivo) {
        biplazasDeportivos.add(biplazaDeportivo);
        registrarOperacion(new RegistroMontaje(new Date(), "Biplaza Deportivo", "Vehículo
terminado"));
        notifyObservadores("Vehículo terminado, Biplaza Deportivo");
    }

    /** {@inheritDoc} */
    @Override
    public List<BiplazaDeportivo> getBiplazasDeportivos() {
        return biplazasDeportivos;
    }

    /**
     * {@inheritDoc}
     * <p>Registra la operación en el historial y notifica al dashboard.</p>
     */
    @Override
    public void agregarFurgoneta(Furgoneta furgoneta) {
        furgonetas.add(furgoneta);
        registrarOperacion(new RegistroMontaje(new Date(), "Furgoneta", "Vehículo
terminado"));
        notifyObservadores("Vehículo terminado, Furgoneta");
    }

    /** {@inheritDoc} */
    @Override
    public List<Furgoneta> getFurgonetas() {
        return furgonetas;
    }

    /**
     * {@inheritDoc}
     * <p>Registra la operación en el historial y notifica al dashboard.</p>
     */
    @Override
    public void agregarTurismo(Turismo turismo) {
        turismos.add(turismo);
        registrarOperacion(new RegistroMontaje(new Date(), "Turismo", "Vehículo
terminado"));
    }

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

        notifyObservadores("Vehículo terminado, Turismo");
    }

    /** {@inheritDoc} */
    @Override
    public List<Turismo> getTurismos() {
        return turismos;
    }

    /**
     * {@inheritDoc}
     * <p>Registra la operación en el historial y notifica al dashboard.</p>
     */
    @Override
    public void agregarMotor(Motor motor) {
        motores.add(motor);
        registrarOperacion(new RegistroMontaje(new Date(), "Motor", "Añadido al almacén"));
        notifyObservadores("Nuevo componente en almacén, Motor " + motor.tipoMotor());
    }

    /** {@inheritDoc} */
    @Override
    public List<Motor> getMotores() {
        return motores;
    }

    /**
     * {@inheritDoc}
     * <p>Registra la operación en el historial y notifica al dashboard.</p>
     */
    @Override
    public void agregarTapiceria(Tapiceria tapiceria) {
        tapicerias.add(tapiceria);
        registrarOperacion(new RegistroMontaje(new Date(), "Tapiceria", "Añadida al
        almacén"));
        notifyObservadores("Nuevo componente en almacén, Tapiceria " +
        tapiceria.tipoTapiceria());
    }

    /** {@inheritDoc} */
    @Override
    public List<Tapiceria> getTapicerias() {
        return tapicerias;
    }

    /**
     * {@inheritDoc}
     * <p>Registra la operación en el historial y notifica al dashboard.</p>

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```
*/
@Override
public void agregarRueda(Rueda rueda) {
    ruedas.add(rueda);
    registrarOperacion(new RegistroMontaje(new Date(), "Rueda", "Añadida al almacén"));
    notifyObservadores("Nuevo componente en almacén, Rueda " + rueda.tipoRueda());
}

/** {@inheritDoc} */
@Override
public List<Rueda> getRuedas() {
    return ruedas;
}

/** {@inheritDoc} */
@Override
public void agregarAdministradorSistema(AdministradorSistema administradorSistema) {
    administradoresSistema.add(administradorSistema);
}

/** {@inheritDoc} */
@Override
public List<AdministradorSistema> getAdministradoresSistema() {
    return administradoresSistema;
}

/** {@inheritDoc} */
@Override
public void agregarGestorPlanta(GestorPlanta gestorPlanta) {
    gestoresPlanta.add(gestorPlanta);
}

/** {@inheritDoc} */
@Override
public List<GestorPlanta> getGestoresPlanta() {
    return gestoresPlanta;
}

/** {@inheritDoc} */
@Override
public void agregarMecanico(Mecanico mecanico) {
    mecanicos.add(mecanico);
}

/** {@inheritDoc} */
@Override
public List<Mecanico> getMecanicos() {
    return mecanicos;
}
```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```
}

/** {@inheritDoc} */
@Override
public void agregarOperario(Operario operario) {
    operarios.add(operario);
}

/** {@inheritDoc} */
@Override
public List<Operario> getOperarios() {
    return operarios;
}

/** {@inheritDoc} */
@Override
public void vaciar() {
    biplazasDeportivos.clear();
    turismos.clear();
    furgonetas.clear();
    ruedas.clear();
    motores.clear();
    tapicerias.clear();
    administradoresSistema.clear();
    gestoresPlanta.clear();
    mecanicos.clear();
    operarios.clear();
    historial.clear();
    observadores.clear();
}

/** {@inheritDoc} */
@Override
public int getTotalCoches() {
    return biplazasDeportivos.size() + turismos.size() + furgonetas.size();
}

/** {@inheritDoc} */
@Override
public void addObserver(Observador observador) {
    observadores.add(observador);
}

/** {@inheritDoc} */
@Override
public void removeObservador(Observador observador) {
    observadores.remove(observador);
}
```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

/** {@inheritDoc} */
@Override
public void notifyObservadores(String message) {
    for (Observador observador : observadores) {
        observador.update(message);
    }
}

/**
 * {@inheritDoc}
 * <p>
 * La comparación de fechas se realiza con precisión de día (mismo año y
 * mismo día del año).
 * </p>
 */
@Override
public List<RegistroMontaje> getRegistrosPorFecha(Date fecha) {
    List<RegistroMontaje> resultado = new ArrayList<>();
    Calendar cal1 = Calendar.getInstance();
    Calendar cal2 = Calendar.getInstance();
    cal1.setTime(fecha);

    for (RegistroMontaje registro : historial) {
        cal2.setTime(registro.getFecha());
        if (cal1.get(Calendar.YEAR) == cal2.get(Calendar.YEAR) &&
            cal1.get(Calendar.DAY_OF_YEAR) == cal2.get(Calendar.DAY_OF_YEAR)) {
            resultado.add(registro);
        }
    }
    return resultado;
}

/** {@inheritDoc} */
@Override
public void registrarOperacion(RegistroMontaje registro) {
    historial.add(registro);
}

/**
 * {@inheritDoc}
 * <p>Si no hay stock, no realiza ninguna acción.</p>
 */
@Override
public void disminuirStockMotor() {

    if(!motores.isEmpty()) {
        motores.remove(0);
    }
}

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```
/**
 * {@inheritDoc}
 * <p>Si no hay stock, no realiza ninguna acción.</p>
 */
@Override
public void disminuirStockRueda() {

    if(!ruedas.isEmpty()) {
        ruedas.remove(0);
    }
}

/**
 * {@inheritDoc}
 * <p>Si no hay stock, no realiza ninguna acción.</p>
 */
@Override
public void disminuirStockTapiceria() {

    if(!tapicerias.isEmpty()) {
        tapicerias.remove(0);
    }
}

/** {@inheritDoc} */
@Override
public int getStockMotores() {

    return motores.size();
}

/** {@inheritDoc} */
@Override
public int getStockRuedas() {

    return ruedas.size();
}

/** {@inheritDoc} */
@Override
public int getStockTapicerias() {

    return tapicerias.size();
}
}
```

Practica de Programación Orientada a Objeto – Curs 2025-2026

ComprobacionStock.java

```
package com.practica.fabrica;

/**
 * Resultado de la comprobación de stock antes de iniciar un ensamblaje.
 * <p>
 * Permite identificar con precisión qué componente tiene stock insuficiente.
 *
 * @author Shao Capilla Sanz
 */
public enum ComprobacionStock {
    /** Stock suficiente para fabricar los vehículos solicitados. */
    OK,
    /** No hay motores suficientes en el almacén. */
    SIN_MOTORES,
    /** No hay tapicerías suficientes en el almacén. */
    SIN_TAPICERIAS,
    /** No hay ruedas suficientes en el almacén. */
    SIN_RUEDAS
}
```

IfaceAlmacen.java

```
package com.practica.fabrica;

import com.practica.vehiculo.*;
import com.practica.rueda.*;
import com.practica.motor.*;
import com.practica.tapiceria.*;
import com.practica.personal.*;
import java.util.*;

/**
 * Contrato del almacén de datos del sistema de gestión de la fábrica.
 * <p>
 * Define las operaciones de alta, consulta y gestión de stock para todas las
 * entidades del sistema (vehículos, componentes, trabajadores y registros
 * históricos de montaje). Su existencia desacopla el sistema de gestión de
 * la estructura de datos concreta, de modo que se pueda sustituir la
 * implementación sin requerir cambios drásticos en el resto del diseño.
 *
 * @author Shao Capilla Sanz
 */
public interface IfaceAlmacen
{

    /**
     * Añade un Biplaza Deportivo terminado al almacén.
     */
}
```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

*
* @param biplazaDeportivo vehículo a almacenar
*/
void agregarBiplazaDeportivo(BiplazaDeportivo biplazaDeportivo);

/**
* @return lista de Biplazas Deportivos terminados almacenados
*/
List<BiplazaDeportivo> getBiplazasDeportivos();

/**
* Añade una Furgoneta terminada al almacén.
*
* @param furgoneta vehículo a almacenar
*/
void agregarFurgoneta(Furgoneta furgoneta);

/**
* @return lista de Furgonetas terminadas almacenadas
*/
List<Furgoneta> getFurgonetas();

/**
* Añade un Turismo terminado al almacén.
*
* @param turismo vehículo a almacenar
*/
void agregarTurismo(Turismo turismo);

/**
* @return lista de Turismos terminados almacenados
*/
List<Turismo> getTurismos();

/**
* Añade un motor al stock del almacén.
*
* @param motor motor a registrar
*/
void agregarMotor(Motor motor);

/**
* @return lista de motores disponibles en stock
*/
List<Motor> getMotores();

/**
* Añade una tapicería al stock del almacén.

```


Practica de Programación Orientada a Objeto – Curs 2025-2026

```

*
* @param tapiceria tapicería a registrar
*/
void agregarTapiceria(Tapiceria tapiceria);

/**
* @return lista de tapicerías disponibles en stock
*/
List<Tapiceria> getTapicerias();

/**
* Añade una rueda al stock del almacén.
*
* @param rueda rueda a registrar
*/
void agregarRueda(Rueda rueda);

/**
* @return lista de ruedas disponibles en stock
*/
List<Rueda> getRuedas();

/**
* Da de alta a un administrador del sistema.
*
* @param administradorSistema administrador a registrar
*/
void agregarAdministradorSistema(AdministradorSistema administradorSistema);

/**
* @return lista de administradores del sistema registrados
*/
List<AdministradorSistema> getAdministradoresSistema();

/**
* Da de alta a un gestor de planta.
*
* @param gestorPlanta gestor a registrar
*/
void agregarGestorPlanta(GestorPlanta gestorPlanta);

/**
* @return lista de gestores de planta registrados
*/
List<GestorPlanta> getGestoresPlanta();

/**
* Da de alta a un mecánico de cinta.

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

*
* @param mecanico mecánico a registrar
*/
void agregarMecanico(Mecanico mecanico);

/**
* @return lista de mecánicos registrados
*/
List<Mecanico> getMecanicos();

/**
* Da de alta a un operario.
*
* @param operario operario a registrar
*/
void agregarOperario(Operario operario);

/**
* @return lista de operarios registrados
*/
List<Operario> getOperarios();

/**
* Vacía todos los datos del almacén (vehículos, stock, trabajadores e
* historial).
*/
void vaciar();

/**
* @return número total de coches terminados almacenados
*/
int getTotalCoches();

/**
* Devuelve los registros de operaciones de montaje correspondientes a
* una fecha concreta (precisión de día).
*
* @param fecha fecha a consultar
* @return lista de registros de esa fecha
*/
List<RegistroMontaje> getRegistrosPorFecha(Date fecha);

/**
* Añade una operación al historial del almacén.
*
* @param registro registro a incorporar
*/
void registrarOperacion(RegistroMontaje registro);

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

/**
 * @return número de motores actualmente en stock
 */
int getStockMotores();

/**
 * @return número de ruedas actualmente en stock
 */
int getStockRuedas();

/**
 * @return número de tapicerías actualmente en stock
 */
int getStockTapicerias();

/**
 * Disminuye en una unidad el stock de motores.
 */
void disminuirStockMotor();

/**
 * Disminuye en una unidad el stock de ruedas.
 */
void disminuirStockRueda();

/**
 * Disminuye en una unidad el stock de tapicerías.
 */
void disminuirStockTapiceria();
}

```

RegistroMontaje.java

```

package com.practica.fabrica;

import java.util.Date;

/**
 * Registro de una operación de montaje realizada en la fábrica.
 * <p>
 * Cada registro captura la fecha y hora en que ocurrió la operación, el tipo
 * de componente involucrado y una descripción de la acción realizada (alta
 * de un componente en el almacén, cambio de estado de un vehículo, etc.).
 * Los registros permiten reconstruir el histórico de actividad y realizar
 * consultas por fecha.
 * @author Shao Capilla Sanz

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

*/
public class RegistroMontaje {
    /** Fecha y hora en que se produjo la operación. */
    private Date fecha;
    /** Tipo de componente o entidad afectada. */
    private String tipoComponente;
    /** Descripción de la acción registrada. */
    private String accion;

    /**
     * Crea un registro de operación de montaje.
     *
     * @param fecha      fecha y hora de la operación
     * @param tipoComponente tipo de componente o entidad afectada
     * @param accion      descripción de la acción
     */
    public RegistroMontaje(Date fecha, String tipoComponente, String accion) {
        this.fecha = fecha;
        this.tipoComponente = tipoComponente;
        this.accion = accion;
    }

    /**
     * @return fecha y hora en que se produjo la operación
     */
    public Date getFecha() {
        return fecha;
    }

    /**
     * Establece la fecha del registro.
     *
     * @param fecha nueva fecha
     */
    public void setFecha(Date fecha) {
        this.fecha = fecha;
    }

    /**
     * @return tipo de componente afectado
     */
    public String getTipoComponente() {
        return tipoComponente;
    }

    /**
     * Establece el tipo de componente del registro.
     *

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

    * @param tipoComponente nuevo tipo de componente
    */
    public void setTipoComponente(String tipoComponente) {
        this.tipoComponente = tipoComponente;
    }

    /**
     * @return descripción de la acción registrada
     */
    public String getAccion() {
        return accion;
    }

    /**
     * Establece la descripción de la acción.
     *
     * @param accion nueva descripción
     */
    public void setAccion(String accion) {
        this.accion = accion;
    }

    /**
     * @return representación textual del registro
     */
    @Override
    public String toString() {
        return "RegistroMontaje{" +
            "fecha=" + fecha +
            ", tipoComponente=" + tipoComponente + "\" +
            ", accion=" + accion + "\" +
            '}'";
    }
}

```

SistemaGestion.java

```

package com.practica.fabrica;

import com.practica.vehiculo.*;
import com.practica.rueda.*;
import com.practica.montaje.CadenaMontaje;
import com.practica.motor.*;
import com.practica.tapiceria.*;
import com.practica.personal.*;

import java.util.ArrayList;

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

import java.util.Comparator;
import java.util.List;
import java.util.HashMap;
import java.util.Map;
import java.util.Date;

/**
 * Fachada del sistema de gestión de la fábrica.
 * <p>
 * Coordina las operaciones de alta y consulta sobre el almacén de datos
 * (representado por {@link IfaceAlmacen}), exponiendo una API de alto nivel
 * para el resto de la aplicación (menús del usuario, planificador, gestor
 * de planta, etc.). Al trabajar contra la interfaz del almacén, su diseño
 * permanece desacoplado de la estructura de datos concreta utilizada. * <p>
 * Responsabilidades:
 * <ul>
 * <li>Alta y consulta de vehículos, componentes y trabajadores.</li>
 * <li>Gestión de stock (motores, tapicerías y ruedas).</li>
 * <li>Búsquedas y listados sobre la plantilla.</li>
 * <li>Estadísticas y consulta del historial de montaje.</li>
 * </ul>
 *
 * @author Shao Capilla Sanz
 */
public class SistemaGestion
{
    /** Almacén de datos contra el que opera el sistema de gestión. */
    private IfaceAlmacen almacen;

    /**
     * Crea un sistema de gestión asociado al almacén indicado.
     *
     * @param almacen implementación del almacén de datos a utilizar
     */
    public SistemaGestion(IfaceAlmacen almacen){
        this.almacen = almacen;
    }

    /**
     * Registra un Biplaza Deportivo terminado en el almacén.
     *
     * @param biplazaDeportivo vehículo a almacenar
     */
    public void registrarBiplazaDeportivo(BiplazaDeportivo biplazaDeportivo){
        almacen.agregarBiplazaDeportivo(biplazaDeportivo);
    }

    /**

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```
* @return lista de Biplazas Deportivos almacenados
*/
public List<BiplazaDeportivo> consultarBiplazasDeportivos(){
    return almacen.getBiplazasDeportivos();
}

/**
 * Registra una Furgoneta terminada en el almacén.
 *
 * @param furgoneta vehículo a almacenar
 */
public void registrarFurgoneta(Furgoneta furgoneta){
    almacen.agregarFurgoneta(furgoneta);
}

/**
 * @return lista de Furgonetas almacenadas
 */
public List<Furgoneta> consultarFurgoneta(){
    return almacen.getFurgonetas();
}

/**
 * Registra un Turismo terminado en el almacén.
 *
 * @param turismo vehículo a almacenar
 */
public void registrarTurismo(Turismo turismo){
    almacen.agregarTurismo(turismo);
}

/**
 * @return lista de Turismos almacenados
 */
public List<Turismo> consultarTurismo(){
    return almacen.getTurismos();
}

/**
 * Añade un motor al stock del almacén.
 *
 * @param motor motor a registrar
 */
public void registrarMotor(Motor motor){
    almacen.agregarMotor(motor);
}

/**
```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```
* @return lista de motores en stock
*/
public List<Motor> consultarMotor(){
    return almacen.getMotores();
}

/**
 * Añade una tapicería al stock del almacén.
 *
 * @param tapiceria tapicería a registrar
 */
public void registrarTapiceria(Tapiceria tapiceria){
    almacen.agregarTapiceria(tapiceria);
}

/**
 * @return lista de tapicerías en stock
 */
public List<Tapiceria> consultarTapiceria(){
    return almacen.getTapicerias();
}

/**
 * Añade una rueda al stock del almacén.
 *
 * @param rueda rueda a registrar
 */
public void registrarRueda(Rueda rueda){
    almacen.agregarRueda(rueda);
}

/**
 * @return lista de ruedas en stock
 */
public List<Rueda> consultarRueda(){
    return almacen.getRuedas();
}

/**
 * Da de alta a un administrador del sistema.
 *
 * @param administradorSistema administrador a registrar
 */
public void registrarAdministradorSistema(AdministradorSistema administradorSistema){
    almacen.agregarAdministradorSistema(administradorSistema);
}

/**
```


Practica de Programación Orientada a Objeto – Curs 2025-2026

```

    * @return lista de administradores del sistema
    */
    public List<AdministradorSistema> consultarAdministradorSistema(){
        return almacen.getAdministradoresSistema();
    }

    /**
     * Da de alta a un gestor de planta.
     *
     * @param gestorPlanta gestor a registrar
     */
    public void registrarGestorPlanta(GestorPlanta gestorPlanta){
        almacen.agregarGestorPlanta(gestorPlanta);
    }

    /**
     * @return lista de gestores de planta
     */
    public List<GestorPlanta> consultarGestorPlanta(){
        return almacen.getGestoresPlanta();
    }

    /**
     * Da de alta a un mecánico de cinta.
     *
     * @param mecanico mecánico a registrar
     */
    public void registrarMecanico(Mecanico mecanico){
        almacen.agregarMecanico(mecanico);
    }

    /**
     * @return lista de mecánicos registrados
     */
    public List<Mecanico> consultarMecanico(){
        return almacen.getMecanicos();
    }

    /**
     * Da de alta a un operario.
     *
     * @param operario operario a registrar
     */
    public void registrarOperario(Operario operario){
        almacen.agregarOperario(operario);
    }

    /**

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

    * @return lista de operarios registrados
    */
    public List<Operario> consultarOperario(){
        return almacen.getOperarios();
    }

    /**
     * Reinicia el sistema vaciando por completo el almacén.
     */
    public void resetSistema(){
        almacen.vaciar();
    }

    /**
     * @return número total de coches almacenados (terminados)
     */
    public int consultarTotalCoches(){
        return almacen.getTotalCoches();
    }

    /**
     * Busca operarios cuyo nombre contiene la cadena indicada (sin distinguir
     * mayúsculas/minúsculas).
     *
     * @param nombre fragmento de nombre a buscar
     * @return lista de operarios coincidentes
     */
    public List<Operario> buscarOperariosPorNombre(String nombre) {
        List<Operario> res = new ArrayList<>();
        String nombreBuscar = nombre.toLowerCase();
        for (Operario op : almacen.getOperarios()) {
            if (op.getNombre().toLowerCase().contains(nombreBuscar)) {
                res.add(op);
            }
        }
        return res;
    }

    /**
     * Devuelve la lista de operarios considerados eficientes (más de 10
     * montajes realizados).
     *
     * @return lista de operarios eficientes
     */
    public List<Operario> buscarOperariosEficientes(){
        List<Operario> res = new ArrayList<>();
        for(Operario op: almacen.getOperarios()){
            if(op.esEficiente()){

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

        res.add(op);
    }
}
return res;
}

/**
 * Busca mecánicos cuyo nombre contiene la cadena indicada (sin distinguir
 * mayúsculas/minúsculas).
 *
 * @param nombre fragmento de nombre a buscar
 * @return lista de mecánicos coincidentes
 */
public List<Mecanico> buscarMecanicosPorNombre(String nombre){
    List<Mecanico> res = new ArrayList<>();
    String nombreBuscar = nombre.toLowerCase();
    for(Mecanico mec: almacen.getMecanicos()){
        if(mec.getNombre().toLowerCase().contains(nombreBuscar)){
            res.add(mec);
        }
    }
    return res;
}

/**
 * Busca un trabajador por su DNI entre todos los perfiles registrados.
 *
 * @param dni DNI a buscar
 * @return cadena descriptiva con el perfil y nombre del trabajador, o
 *         "No encontrado" si no existe
 */
public String buscarTrabajadorPorDni(String dni) {
    for (Operario op : almacen.getOperarios()) {
        if (op.getDni().equals(dni)) return "Operario: " + op.getNombre() + " " +
op.getApellidos();
    }
    for (Mecanico mec : almacen.getMecanicos()) {
        if (mec.getDni().equals(dni)) return "Mecánico: " + mec.getNombre() + " " +
mec.getApellidos();
    }
    for (GestorPlanta gp : almacen.getGestoresPlanta()) {
        if (gp.getDni().equals(dni)) return "Gestor: " + gp.getNombre() + " " +
gp.getApellidos();
    }
    for (AdministradorSistema as : almacen.getAdministradoresSistema()) {
        if (as.getDni().equals(dni)) return "Administrador: " + as.getNombre() + " " +
as.getApellidos();
    }
}

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

        return "No encontrado";
    }

    /**
     * Comprueba si hay stock suficiente para fabricar el número de vehículos
     * indicado. Cada vehículo requiere un motor, una tapicería y cuatro
     * ruedas; si falta cualquier componente, muestra el motivo por consola.
     *
     * @param cantidad número de vehículos que se desean fabricar
     * @return {@code true} si hay stock suficiente; {@code false} en otro caso
     */
    public ComprobacionStock comprobarStock(int cantidad){
        int cantidadMotores = almacen.getStockMotores();
        int cantidadTapiceria = almacen.getStockTapicerias();
        int cantidadRueda = almacen.getStockRuedas();

        if (cantidadMotores < cantidad) {
            return ComprobacionStock.SIN_MOTORES;
        }
        if (cantidadTapiceria < cantidad) {
            return ComprobacionStock.SIN_TAPICERIAS;
        }
        if (cantidadRueda < cantidad * 4) {
            return ComprobacionStock.SIN_RUEDAS;
        }
        return ComprobacionStock.OK;
    }

    /**
     * Extrae un motor del stock, eliminándolo del almacén.
     *
     * @return motor extraído o {@code null} si no había stock
     */
    public Motor quitarStockMotor() {
        List<Motor> motores = almacen.getMotores();
        if (motores.isEmpty()) {
            return null;
        }
        Motor m = motores.get(0);
        almacen.disminuirStockMotor();
        return m;
    }

    /**
     * Extrae una tapicería del stock, eliminándola del almacén.
     *
     * @return tapicería extraída o {@code null} si no había stock
     */

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

public Tapiceria quitarStockTapiceria() {
    List<Tapiceria> tapicerias = almacen.getTapicerias();
    if (tapicerias.isEmpty()) {
        return null;
    }
    Tapiceria t = tapicerias.get(0);
    almacen.disminuirStockTapiceria();
    return t;
}

/**
 * Extrae del stock un juego de 4 ruedas. Si es posible, las 4 ruedas
 * extraídas pertenecen al mismo tipo (Normal, Deportivo o Todoterreno);
 * en caso contrario se entregan las primeras 4 disponibles del stock.
 *
 * @return array de 4 ruedas o {@code null} si no había stock suficiente
 */
public Rueda[] quitarStockRuedas() {
    List<Rueda> ruedas = almacen.getRuedas();
    if (ruedas.size() < 4) {
        return null;
    }

    String tipoBuscado = null;
    int contadas = 0;
    for (Rueda rTipo : ruedas) {
        if (tipoBuscado == null || !rTipo.tipoRueda().equals(tipoBuscado)) {
            tipoBuscado = rTipo.tipoRueda();
            contadas = 1;
        } else {
            contadas++;
        }
        if (contadas == 4) break;
    }

    Rueda[] r = new Rueda[4];
    if (contadas == 4 && tipoBuscado != null) {

        int asignadas = 0;
        int i = 0;
        while (asignadas < 4 && i < ruedas.size()) {
            if (ruedas.get(i).tipoRueda().equals(tipoBuscado)) {
                r[asignadas++] = ruedas.remove(i);
            } else {
                i++;
            }
        }
    } else {

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

        for (int i = 0; i < 4; i++) {
            r[i] = ruedas.get(0);
            almacen.disminuirStockRueda();
        }
    }
    return r;
}

/**
 * Crea y registra en el historial del almacén una nueva entrada con la
 * fecha actual.
 *
 * @param tipoComponente tipo de componente o entidad afectada
 * @param accion descripción de la acción
 */
public void registrarHistorial(String tipoComponente, String accion) {
    almacen.registrarOperacion(new RegistroMontaje(new Date(), tipoComponente,
accion));
}

/**
 * Devuelve la lista de operarios ordenada alfabéticamente por nombre y,
 * en caso de empate, por apellidos.
 *
 * @return lista de operarios ordenada
 */
public List<Operario> ordenarOperarios(){
    List<Operario> res = new ArrayList<>(almacen.getOperarios());

    res.sort(Comparator.comparing(Trabajador::getNombre).thenComparing(Trabajador::getAp
ellidos));
    return res;
}

/**
 * Devuelve la lista de operarios cuya productividad (montajes realizados)
 * iguala o supera un umbral mínimo, ordenada de mayor a menor productividad.
 *
 * @param minMontajes umbral mínimo de montajes realizados
 * @return lista filtrada y ordenada por productividad descendente
 */
public List<Operario> obtenerOperariosProductividad(int minMontajes) {
    List<Operario> res = new ArrayList<>();
    for (Operario op : almacen.getOperarios()) {
        if (op.getMontajesRealizados() >= minMontajes) {
            res.add(op);
        }
    }
}

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

        res.sort(Comparator.comparingInt(Operario::getMontajesRealizados).reversed());
        return res;
    }

    /**
     * Devuelve todos los coches terminados (Biplazas, Turismos y Furgonetas)
     * almacenados en el sistema.
     *
     * @param cadenaMontaje cadena de montaje (no consultada en esta
     * implementación; se mantiene por compatibilidad)
     * @return lista completa de coches terminados
     */
    public List<Coche> obtenerCochesTerminados(CadenaMontaje cadenaMontaje) {
        List<Coche> res = new ArrayList<>();
        res.addAll(consultarBiplazasDeportivos());
        res.addAll(consultarTurismo());
        res.addAll(consultarFurgoneta());
        return res;
    }

    /**
     * Filtra una lista de coches devolviendo únicamente los que tienen el
     * tipo de motor indicado (comparación sin distinguir mayúsculas).
     *
     * @param coches lista de coches a filtrar
     * @param tipoMotor tipo de motor buscado (p.ej. "Gasolina")
     * @return lista filtrada
     */
    public List<Coche> filtrarTipoMotor(List<Coche> coches, String tipoMotor) {
        List<Coche> res = new ArrayList<>();
        for (Coche c: coches) {
            if(c.getMotor() != null && c.getMotor().tipoMotor().equalsIgnoreCase(tipoMotor)) {
                res.add(c);
            }
        }
        return res;
    }

    /**
     * Filtra una lista de coches devolviendo únicamente los que tienen el
     * tipo de tapicería indicado (comparación sin distinguir mayúsculas).
     *
     * @param coches lista de coches a filtrar
     * @param tipoTapiceria tipo de tapicería buscado (p.ej. "Cuero")
     * @return lista filtrada
     */
    public List<Coche> filtrarTipoTapiceria(List<Coche> coches, String tipoTapiceria) {
        List<Coche> res = new ArrayList<>();
    }

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

        for (Coche c: coches){
            if(c.getTapiceria() != null &&
c.getTapiceria().tipoTapiceria().equalsIgnoreCase(tipoTapiceria)) {
                res.add(c);
            }
        }
        return res;
    }

    /**
     * Devuelve los coches indicados ordenados alfabéticamente por tipo de
     * vehículo.
     *
     * @param coches lista de coches a ordenar
     * @return lista ordenada por tipo de coche
     */
    public List<Coche> obtenerCochesOrdenados(List<Coche> coches) {
        List<Coche> res = new ArrayList<>(coches);
        res.sort(Comparator.comparing(Coche::tipoCoche));
        return res;
    }

    /**
     * Calcula las configuraciones más ensambladas a partir de los coches
     * terminados. La configuración se identifica por la combinación de tipo
     * de vehículo, motor, tapicería y rueda.
     *
     * @param cadenaMontaje cadena de montaje (no consultada en esta
     * implementación; se mantiene por compatibilidad)
     * @return mapa cuya clave es la configuración y cuyo valor es el número
     * de unidades ensambladas con esa configuración
     */
    public Map<String, Integer> getConfiguracionesMasEnsamblados(CadenaMontaje
cadenaMontaje) {
        Map<String, Integer> res = new HashMap<>();
        List<Coche> cochesTerminados = obtenerCochesTerminados(cadenaMontaje);
        for (Coche c : cochesTerminados) {
            String config = c.tipoCoche()
                + " | Motor: " + (c.getMotor() != null ? c.getMotor().tipoMotor() : "N/A")
                + " | Tapicería: " + (c.getTapiceria() != null ? c.getTapiceria().tipoTapiceria() : "N/A")
                + " | Rueda: " + (c.getRueda() != null && c.getRueda().length > 0 ?
c.getRueda()[0].tipoRueda() : "N/A");
            res.put(config, res.getOrDefault(config, 0) + 1);
        }
        return res;
    }

    /**

```


Practica de Programación Orientada a Objeto – Curs 2025-2026

```

    * Devuelve los registros de operaciones de montaje correspondientes a la
    * fecha indicada (precisión de día).
    *
    * @param fecha fecha a consultar
    * @return lista de registros encontrados
    */
    public List<RegistroMontaje> consultarHistorialFecha(Date fecha) {
        return almacen.getRegistrosPorFecha(fecha);
    }
}

```

7.8 com.practica.montaje

CadenaMontaje.java

```

package com.practica.montaje;

import com.practica.vehiculo.*;
import com.practica.dashboard.*;
import java.util.ArrayList;

/**
 * Cadena de montaje de la fábrica de vehículos.
 * <p>
 * Aglutina las tres líneas de producción de la factoría, una por cada tipo de
 * vehículo del catálogo (Biplaza Deportivo, Turismo y Furgoneta), y mantiene
 * la cola de unidades pendientes o en curso para cada línea. * <p>
 * Implementa el patrón Observador como sujeto observable, de modo que cada
 * cambio relevante (incorporación de un vehículo, cambio de estado, etc.) se
 * notifica a los observadores registrados (típicamente el dashboard). *
 * @author Shao Capilla Sanz
 */
public class CadenaMontaje implements Observable {

    /** Cola de Biplazas Deportivos en montaje. */
    private ArrayList<BiplazaDeportivo> cadenaBiplaza;
    /** Cola de Turismos en montaje. */
    private ArrayList<Turismo> cadenaTurismo;
    /** Cola de Furgonetas en montaje. */
    private ArrayList<Furgoneta> cadenaFurgoneta;
    /** Lista de observadores registrados para recibir notificaciones. */
    private ArrayList<Observador> observadores;

    /**
     * Construye una cadena de montaje vacía, con las tres líneas y la lista
     * de observadores inicializadas.
     */
}

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

*/
public CadenaMontaje() {

    cadenaBiplaza = new ArrayList<BiplazaDeportivo>();
    cadenaTurismo = new ArrayList<Turismo>();
    cadenaFurgoneta = new ArrayList<Furgoneta>();
    observadores = new ArrayList<Observador>();
}

/**
 * Añade un Biplaza Deportivo a su línea de montaje y notifica a los
 * observadores.
 *
 * @param biplaza vehículo a encolar
 */
public void agregarBiplaza(BiplazaDeportivo biplaza) {
    cadenaBiplaza.add(biplaza);
    notifyObservadores("Se ha agregado un Biplaza Deportivo a la cadena de montaje.");
}

/**
 * Añade un Turismo a su línea de montaje y notifica a los observadores.
 *
 * @param turismo vehículo a encolar
 */
public void agregarTurismo(Turismo turismo) {
    cadenaTurismo.add(turismo);
    notifyObservadores("Se ha agregado un Turismo a la cadena de montaje.");
}

/**
 * Añade una Furgoneta a su línea de montaje y notifica a los observadores.
 *
 * @param furgoneta vehículo a encolar
 */
public void agregarFurgoneta(Furgoneta furgoneta) {
    cadenaFurgoneta.add(furgoneta);
    notifyObservadores("Se ha agregado una Furgoneta a la cadena de montaje.");
}

/**
 * @return lista de Biplazas Deportivos en la cadena
 */
public ArrayList<BiplazaDeportivo> getCadenaBiplaza() {
    return cadenaBiplaza;
}

/**

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

    * @return lista de Turismos en la cadena
    */
    public ArrayList<Turismo> getCadenaTurismo() {
        return cadenaTurismo;
    }

    /**
     * @return lista de Furgonetas en la cadena
     */
    public ArrayList<Furgoneta> getCadenaFurgoneta() {
        return cadenaFurgoneta;
    }

    /**
     * Vacía por completo las tres líneas de montaje.
     */
    public void limpiar() {
        cadenaBiplaza.clear();
        cadenaTurismo.clear();
        cadenaFurgoneta.clear();
    }

    /**
     * Elimina de las tres líneas todos los vehículos que ya se encuentran en
     * estado {@link EstadoMontaje#TERMINADO}, dejando únicamente los que
     * siguen pendientes de ensamblar.
     *
     * @return número total de vehículos purgados
     */
    public int purgarTerminados() {
        int antesB = cadenaBiplaza.size();
        int antesT = cadenaTurismo.size();
        int antesF = cadenaFurgoneta.size();
        cadenaBiplaza.removeIf(c -> c.getEstadoMontaje() == EstadoMontaje.TERMINADO);
        cadenaTurismo.removeIf(c -> c.getEstadoMontaje() == EstadoMontaje.TERMINADO);
        cadenaFurgoneta.removeIf(c -> c.getEstadoMontaje() == EstadoMontaje.TERMINADO);
        return (antesB - cadenaBiplaza.size()) + (antesT - cadenaTurismo.size()) + (antesF -
        cadenaFurgoneta.size());
    }

    /**
     * {@inheritDoc}
     */
    @Override
    public void addObserver(Observador observador) {
        observadores.add(observador);
    }

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

/**
 * {@inheritDoc}
 */
@Override
public void removeObservador(Observador observador) {
    observadores.remove(observador);
}

/**
 * {@inheritDoc}
 */
@Override
public void notifyObservadores(String message) {
    for (Observador ob : observadores) {
        ob.update(message);
    }
}
}

```

Robot.java

```

package com.practica.montaje;

import com.practica.personal.Operario;
import com.practica.vehiculo.Coche;

/**
 * Robot de montaje controlado por un operario en una estación de la cadena.
 * <p>
 * Cada robot está especializado en un componente concreto (chasis, motor,
 * tapicería o ruedas) y trabaja sobre un único coche a la vez. El tiempo que
 * necesita para completar el montaje depende del perfil (eficiente o estándar)
 * del operario que lo controla.
 * @author Shao Capilla Sanz
 */
public class Robot {
    /** Operario que controla el robot. */
    private Operario operario;
    /** Coche que se está ensamblando actualmente; {@code null} si está libre. */
    private Coche cocheActual;
    /** Segundos de simulación que restan para terminar el montaje en curso. */
    private int tiempoRestante;
    /** Nombre identificativo del robot. */
    private String nombre;

    /**
     * Crea un robot con un nombre identificativo y el operario que lo controla.

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

*
* @param nombre nombre del robot
* @param operario operario asignado al robot
*/
public Robot(String nombre, Operario operario) {
    this.nombre = nombre;
    this.operario = operario;
    this.tiempoRestante = 0;
    this.cocheActual = null;
}

/**
* @return {@code true} si el robot no está trabajando con ningún coche
*/
public boolean estaLibre() {
    return cocheActual == null;
}

/**
* Asigna un coche al robot para que comience el montaje. El tiempo de
* trabajo se inicializa al tiempo de montaje del operario asignado.
*
* @param c coche a ensamblar
*/
public void recibirCoche(Coche c) {
    this.cocheActual = c;
    this.tiempoRestante = operario.getTiempoMontaje();
}

/**
* @return coche que el robot está ensamblando actualmente, o {@code null}
*/
public Coche getCocheActual() {
    return cocheActual;
}

/**
* Avanza un segundo de simulación el trabajo del robot. Si el coche en
* curso es {@code null} no realiza ninguna acción. Cuando el tiempo
* restante alcanza cero, registra el montaje completado en el operario y
* comunica que ha finalizado.
*
* @return {@code true} si el montaje ha terminado en este tick;
*         {@code false} en caso contrario
*/
public boolean trabajar() {
    if (cocheActual == null) {
        return false;
    }

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

    }

    tiempoRestante--;
    if (tiempoRestante <= 0) {
        operario.registrarMontajeCompletado();
        return true;
    }
    return false;
}

/**
 * Libera al robot del coche actual para que pueda recibir uno nuevo.
 */
public void liberarCoche() {
    this.cocheActual = null;
}

/**
 * @return nombre identificador del robot
 */
public String getNombre() {
    return nombre;
}
}

```

7.9 com.practica.planificador

Planificador.java

```

package com.practica.planificador;

import com.practica.dashboard.*;
import com.practica.fabrica.*;
import com.practica.montaje.*;
import com.practica.vehiculo.*;
import com.practica.motor.*;
import com.practica.tapiceria.*;
import com.practica.rueda.*;
import com.practica.personal.*;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

/**
 * Motor de simulación de la fábrica.

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

* <p>
* El planificador orquesta el avance del montaje en las tres cadenas
* (Biplaza, Turismo y Furgoneta) tick a tick (1 segundo real), asignando
* los robots de cada estación a los vehículos y consumiendo stock del
* almacén a medida que se progresa por los estados de
* {@link EstadoMontaje}. Es el único componente que avanza el tiempo de
* la simulación. * <p>
* Soporta tres niveles de simulación:
* <ul>
* <li><b>Simple (1)</b>: sin incidencias.</li>
* <li><b>Compleja (2)</b>: averías de vehículos que requieren mecánico.</li>
* <li><b>Muy compleja (3)</b>: averías + apagón general que paraliza la
*   fábrica hasta que el administrador restaure el sistema de gestión
*   (2 s) y las cadenas (3 s).</li>
* </ul> * <p>
* Implementa {@link Observable} para poder notificar eventos relevantes
* al {@link Dashboard} y a otros observadores. *
* @author Shao Capilla Sanz
*/
public class Planificador implements Observable {

    /** Cadena de montaje sobre la que se ejecuta la simulación. */
    private CadenaMontaje cadenaMontaje;
    /** Sistema de gestión que mantiene almacén y registros. */
    private SistemaGestion sistemaGestion;
    /** Nivel de simulación: 1=Simple, 2=Compleja, 3=Muy compleja. */
    private int tipoSimulacion;

    /** Indica si la fábrica está en apagón. */
    private boolean caidaDeLuz = false;
    /** Indica si el sistema de gestión está bloqueado por el apagón. */
    private boolean sistemaGestionBloqueado = false;
    /** Segundos restantes para restaurar el sistema de gestión. */
    private int tiempoReparacionGestion = 0;
    /** Segundos restantes para restaurar las cadenas de montaje. */
    private int tiempoReparacionCadenas = 0;
    /** Marca si ya se ha generado el apagón en esta simulación. */
    private boolean apagonGenerado = false;

    /** Número de averías generadas en la cadena Biplaza. */
    private int averiasBiplaza = 0;
    /** Número de averías generadas en la cadena Turismo. */
    private int averiasTurismo = 0;
    /** Número de averías generadas en la cadena Furgoneta. */
    private int averiasFurgoneta = 0;

    /** Lista de observadores suscritos al planificador. */
    private ArrayList<Observador> observadores;

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

/** Robots de las 4 estaciones de la cadena Biplaza. */
private Robot[] robotsBiplaza = new Robot[4];
/** Robots de las 4 estaciones de la cadena Turismo. */
private Robot[] robotsTurismo = new Robot[4];
/** Robots de las 4 estaciones de la cadena Furgoneta. */
private Robot[] robotsFurgoneta = new Robot[4];

/** Mecánicos disponibles para resolver averías (uno por cadena). */
private Mecanico[] mecanicos;
/** Administrador de sistema responsable de restaurar tras el apagón. */
private AdministradorSistema admin;
/** Gestor de planta encargado de avisos y llamadas al mecánico. */
private GestorPlanta gestorPlanta;

/** Dashboard al que el planificador notifica avisos y resúmenes de simulación. */
private Dashboard dashboard;

/**
 * Construye un planificador con los recursos necesarios para simular la
 * fábrica. Asocia operarios reales a los robots si los hay; en caso
 * contrario genera operarios sintéticos para cubrir las 4 estaciones de
 * las 3 cadenas (12 operarios).
 *
 * @param cadenaMontaje cadena de montaje sobre la que operar
 * @param sistemaGestion sistema de gestión para stock e historial
 * @param tipoSimulacion nivel de simulación (1, 2 o 3)
 * @param mecanicos mecánicos disponibles
 * @param admin administrador del sistema (puede ser {@code null}
 * en niveles que no lo requieran)
 * @param dashboard dashboard al que se enviarán avisos y resúmenes
 */
public Planificador(CadenaMontaje cadenaMontaje, SistemaGestion sistemaGestion, int
tipoSimulacion, Mecanico[] mecanicos, AdministradorSistema admin, Dashboard
dashboard) {
    this.cadenaMontaje = cadenaMontaje;
    this.sistemaGestion = sistemaGestion;
    this.tipoSimulacion = tipoSimulacion;
    this.mecanicos = mecanicos;
    this.admin = admin;
    this.dashboard = dashboard;

    List<GestorPlanta> gestores = sistemaGestion.consultarGestorPlanta();
    this.gestorPlanta = gestores.isEmpty() ? null : gestores.get(0);
    this.observadores = new ArrayList<>();

    String[] componentes = { "Chasis", "Motor", "Tapicería", "Ruedas" };
    List<Operario> registrados = sistemaGestion.consultarOperario();

```


Practica de Programación Orientada a Objeto – Curs 2025-2026

```

int indice = 0;
for (int i = 0; i < 4; i++) {
    robotsBiplaza[i] = new Robot("Robot Biplaza" + componentes[i],
        obtenerOperario(registrados, indice++, "Biplaza", i));
    robotsTurismo[i] = new Robot("Robot Turismo" + componentes[i],
        obtenerOperario(registrados, indice++, "Turismo", i));
    robotsFurgoneta[i] = new Robot("Robot Furgoneta" + componentes[i],
        obtenerOperario(registrados, indice++, "Furgoneta", i));
}
}

/**
 * Devuelve un operario para la cadena/estación indicada. Si hay operarios
 * registrados, los recorre en orden circular; si no, genera uno sintético.
 *
 * @param registrados lista de operarios registrados
 * @param indice índice de asignación
 * @param cadena nombre de la cadena (Biplaza/Turismo/Furgoneta)
 * @param posicion posición dentro de la cadena (0-3)
 * @return operario asignado
 */
private Operario obtenerOperario(List<Operario> registrados, int indice, String cadena, int
posicion) {
    if (!registrados.isEmpty()) {
        return registrados.get(indice % registrados.size());
    }
    return crearOperarioAleatorio(cadena, posicion);
}

/**
 * Crea un operario sintético con datos aleatorios cuando no hay
 * operarios registrados. La mitad de las veces el operario es eficiente
 * (más de 10 montajes ya realizados).
 *
 * @param nombreCadena nombre de la cadena para componer el nombre
 * @param indice índice de la estación
 * @return operario generado
 */
private Operario crearOperarioAleatorio(String nombreCadena, int indice) {

    String sufijo = nombreCadena + "_" + indice + "_" + System.nanoTime();
    String nombre = "Operario_" + nombreCadena + "_" + indice;
    String apellidos = "Apellido_" + sufijo;
    String dni = String.format("%08d", Math.abs(sufijo.hashCode()) % 100000000) + "A";
    String direccion = "Calle " + sufijo;
    String seguridadSocial = "Seguridad Social_" + sufijo;
    double salario = 25000;
    LocalDate fechaIngreso = LocalDate.now();

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```
Operario op = new Operario(nombre, apellidos, dni, direccion, seguridadSocial, salario,
fechaIngreso);
```

```
boolean esEficiente = Math.random() < 0.5;
if (esEficiente) {
    for (int j = 0; j < 11; j++) {
        op.registrarMontajeCompletado();
    }
}
```

```
return op;
}
```

```
/**
```

```
* Inicia la simulación y avanza segundo a segundo hasta que todas las
* cadenas hayan terminado o se alcance el tope de 500 ticks. En cada
```

```
* tick:
```

```
* <ol>
```

```
* <li>Resuelve incidencias pendientes (apagón y averías).</li>
```

```
* <li>Avanza los vehículos por las estaciones libres.</li>
```

```
* <li>Genera nuevos eventos (averías y apagón).</li>
```

```
* <li>Comprueba si la fabricación ha terminado.</li>
```

```
* </ol>
```

```
* Al finalizar imprime un resumen de la actividad de los trabajadores.
```

```
*/
```

```
public void iniciarSimulacion() {
```

```
    int purgados = cadenaMontaje.purgarTerminados();
    if (purgados > 0) {
        System.out.println("[INFO] Purgados " + purgados
            + " vehículos ya terminados de la cadena activa.");
    }
}
```

```
    int segundo = 1;
```

```
    int maxTicks = 500;
```

```
    boolean montajeTerminado = false;
```

```
    while (!montajeTerminado && segundo <= maxTicks) {
        System.out.println("\n--- T=" + segundo + " s ---");
        resolverIncidencias();
        trabajarEnEstaciones();
        generarEventos(segundo);
        try {
            Thread.sleep(1000);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

        segundo++;
        montajeTerminado = comprobarFinMontaje();
    }
    if (segundo > maxTicks) {
        dashboard.mostrarError("\n[ABORTADA] Se alcanzó el tope de " + maxTicks
            + " segundos. Revise mecánicos / stock.");
    } else {
        notifyObservadores("\nSimulación terminada en " + (segundo - 1) + " segundos.");
    }
    imprimirResumenTrabajadores();
}

/**
 * Hace trabajar a todas las cadenas durante un tick. Si hay un apagón,
 * todas las cadenas quedan paradas.
 */
private void trabajarEnEstaciones() {

    if (caidaDeLuz) {
        System.out.println("[APAGÓN] Sistema de gestión bloqueado: " +
            sistemaGestionBloqueado + ". Cadenas paradas.");
        return;
    }

    procesarCadenaConRobots(cadenaMontaje.getCadenaBiplaza(), robotsBiplaza, "Biplaza");

    procesarCadenaConRobots(cadenaMontaje.getCadenaTurismo(), robotsTurismo, "Turismo");

    procesarCadenaConRobots(cadenaMontaje.getCadenaFurgoneta(), robotsFurgoneta, "Furgo
neta");
}

/**
 * Avanza una unidad de tiempo en una cadena concreta. Para cada coche:
 * <ul>
 * <li>Si está terminado o averiado, lo salta.</li>
 * <li>Selecciona el robot que corresponde a su estado actual.</li>
 * <li>Si el robot está libre, le entrega el coche; si está ocupado con
 * otro coche, espera al siguiente tick.</li>
 * <li>Cuando el robot completa su trabajo, consume el componente
 * correspondiente del stock, lo monta en el coche y avanza al
 * siguiente estado. Si llega a TERMINADO, registra el vehículo
 * en el almacén.</li>
 * </ul>
 *
 * @param lista lista de coches de la cadena
 * @param robots robots de las 4 estaciones de la cadena
 * @param nombreCadena nombre legible de la cadena (para logs)

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```
*/
private void procesarCadenaConRobots(ArrayList<? extends Coche> lista, Robot[] robots,
String nombreCadena) {

    int i = 0;
    for (Coche c : lista) {
        if (c.getEstadoMontaje() == EstadoMontaje.TERMINADO) {
            i++;
            continue;
        }
        if (c.isAveriado()) {
            notifyObservadores("[AVISO] Coche " + nombreCadena + " #" + (i + 1) + " averiado -
esperando mecánico.");
            i++;
            continue;
        }

        EstadoMontaje estadoActual = c.getEstadoMontaje();
        if (estadoActual == null) {
            estadoActual = EstadoMontaje.CHASIS;
            c.setEstadoMontaje(EstadoMontaje.CHASIS);
        }

        int indexRobot;
        EstadoMontaje estadoSiguiente;

        switch (estadoActual) {
            case CHASIS:
                indexRobot = 0;
                estadoSiguiente = EstadoMontaje.MOTOR;
                break;
            case MOTOR:
                indexRobot = 1;
                estadoSiguiente = EstadoMontaje.TAPICERIA;
                break;
            case TAPICERIA:
                indexRobot = 2;
                estadoSiguiente = EstadoMontaje.RUEDAS;
                break;
            case RUEDAS:
                indexRobot = 3;
                estadoSiguiente = EstadoMontaje.TERMINADO;
                break;
            default:
                i++;
                continue;
        }
    }
}
```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

Robot robotAsignado = robots[indexRobot];

if (robotAsignado.getCocheActual() != c) {
    if (robotAsignado.estaLibre()) {
        robotAsignado.recibirCoche(c);
    } else {
        i++;
        continue;
    }
}

boolean terminado = robotAsignado.trabajar();
if (terminado) {
    String detalleComponente = "";
    switch (estadoSiguiente) {
        case MOTOR: {
            Motor m = sistemaGestion.quitarStockMotor();
            if (m != null) {
                c.setMotor(m);
                detalleComponente = " | Motor=" + m.tipoMotor() + " (" + m.getCilindrada()
+ "cc, " + m.getPotencia() + "CV)";
            }
            break;
        }
        case TAPICERIA: {
            Tapiceria t = sistemaGestion.quitarStockTapiceria();
            if (t != null) {
                c.setTapiceria(t);
                detalleComponente = " | Tapicería=" + t.tipoTapiceria() + " (" + t.getColor() +
+ "mm)";
            }
            break;
        }
        case RUEDAS: {
            Rueda[] r = sistemaGestion.quitarStockRuedas();
            if (r != null) {
                c.setRueda(r);
                detalleComponente = " | Ruedas=" + r[0].tipoRueda() + " (" + r[0].getAncho()
+ "mm)";
            }
            break;
        }
        default:
            break;
    }

    c.setEstadoMontaje(estadoSiguiente);
    robotAsignado.liberarCoche();
}

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

        if (estadoSiguiente == EstadoMontaje.TERMINADO) {
            if (c instanceof BiplazaDeportivo) {
                sistemaGestion.registrarBiplazaDeportivo((BiplazaDeportivo) c);
            } else if (c instanceof Turismo) {
                sistemaGestion.registrarTurismo((Turismo) c);
            } else if (c instanceof Furgoneta) {
                sistemaGestion.registrarFurgoneta((Furgoneta) c);
            }
        }

        sistemaGestion.registrarHistorial(nombreCadena + " #" + (i + 1), estadoActual + " -> " + estadoSiguiente + detalleComponente);

        String msg = "[" + nombreCadena + " #" + (i + 1) + "]" + estadoActual + " -> " + estadoSiguiente + detalleComponente;
        cadenaMontaje.notifyObservadores(msg);
    }
    i++;
}
}

/**
 * Genera eventos aleatorios (averías y apagón) según el nivel de
 * simulación. Las averías iniciales se producen de forma determinista
 * en los primeros segundos; a partir de la tercera avería pueden
 * surgir nuevas con probabilidad del 15% por tick (hasta un máximo de
 * 2 ó 3 según el nivel). En nivel 3, a partir del segundo 3 se genera
 * un apagón único que bloquea la fábrica.
 *
 * @param segundo tick actual de la simulación
 */
private void generarEventos(int segundo) {
    if (tipoSimulacion == 1) return;

    if (averiasBiplaza < 2 && segundo == 1 + averiasBiplaza * 2) {
        if (generarAveriaLista(cadenaMontaje.getCadenaBiplaza(), "Biplaza")) {
            averiasBiplaza++;
        }
    }

    if (averiasTurismo < 2 && segundo == 1 + averiasTurismo * 2) {
        if (generarAveriaLista(cadenaMontaje.getCadenaTurismo(), "Turismo")) {
            averiasTurismo++;
        }
    }

    if (averiasFurgoneta < 2 && segundo == 1 + averiasFurgoneta * 2) {
        if (generarAveriaLista(cadenaMontaje.getCadenaFurgoneta(), "Furgoneta")) {
            averiasFurgoneta++;
        }
    }
}

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

    }
}

int maxAverias = (tipoSimulacion == 3) ? 3 : 2;
if (averiasBiplaza < maxAverias && averiasBiplaza >= 2 && Math.random() < 0.15) {
    if (generarAveriaLista(cadenaMontaje.getCadenaBiplaza(), "Biplaza")) {
        averiasBiplaza++;
    }
}
if (averiasTurismo < maxAverias && averiasTurismo >= 2 && Math.random() < 0.15) {
    if (generarAveriaLista(cadenaMontaje.getCadenaTurismo(), "Turismo")) {
        averiasTurismo++;
    }
}
if (averiasFurgoneta < maxAverias && averiasFurgoneta >= 2 && Math.random() < 0.15)
{
    if (generarAveriaLista(cadenaMontaje.getCadenaFurgoneta(), "Furgoneta")) {
        averiasFurgoneta++;
    }
}

if (tipoSimulacion == 3 && lapagonGenerado && segundo >= 3) {
    caidaDeLuz = true;
    sistemaGestionBloqueado = true;
    tiempoReparacionGestion = 2;
    tiempoReparacionCadenas = 3;
    apagonGenerado = true;
    String msg = "La luz se ha caído - El Administrador trabajará para restaurarla.";
    cadenaMontaje.notifyObservadores(msg);
}
}

/**
 * Avería el primer coche no terminado y no averiado de la lista. Marca
 * el tiempo de reparación como pendiente de asignación (-1) y notifica
 * al gestor de planta.
 *
 * @param lista lista de coches candidatos a averiarse
 * @param nombreCadena nombre legible de la cadena
 * @return {@code true} si se averió un coche; {@code false} si no había
 * candidatos
 */
private boolean generarAveriaLista(ArrayList<? extends Coche> lista, String
nombreCadena) {
    for (Coche c : lista) {
        if (c.getEstadoMontaje() != EstadoMontaje.TERMINADO && !c.isAveriado()) {
            c.setAveriado(true);
            c.setTiempoReparacion(-1);

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

        if (gestorPlanta != null) {
            gestorPlanta.consultarDashboard();
        }

        String quien = (gestorPlanta != null) ? "El Gestor de Planta " +
gestorPlanta.getNombre() + " " + gestorPlanta.getApellidos() : "El Gestor de Planta";
        String msg = "Se ha detectado avería en cadena " + nombreCadena + " - " + quien +
" llamará al mecánico.";
        cadenaMontaje.notifyObservadores(msg);
        return true;
    }
}
return false;
}

/**
 * Resuelve incidencias activas en el tick actual: avance del tiempo de
 * restauración del apagón y reparación progresiva de cada coche
 * averiado por su mecánico asignado.
 */
private void resolverIncidencias() {
    if (tipoSimulacion == 1) {
        return;
    }

    if (tipoSimulacion == 3 && caidaDeLuz) {

        if (sistemaGestionBloqueado) {
            tiempoReparacionGestion--;
            if (tiempoReparacionGestion <= 0 && admin != null) {
                admin.restaurarSistemaGestion(this);
            }
        }
        tiempoReparacionCadenas--;
        if (tiempoReparacionCadenas <= 0 && admin != null) {
            admin.restaurarCadenasMontaje(this);
        }
    }

    resolverAveriasConMecanico(cadenaMontaje.getCadenaBiplaza(), 0);
    resolverAveriasConMecanico(cadenaMontaje.getCadenaTurismo(), 1);
    resolverAveriasConMecanico(cadenaMontaje.getCadenaFurgoneta(), 2);
}

/**
 * Para cada coche averiado de la lista, decrementa su contador de
 * reparación según el mecánico asignado. Cuando el contador llega a 0,

```


Practica de Programación Orientada a Objeto – Curs 2025-2026

```

    * el gestor llama al mecánico para que repare el coche y se emite el
    * aviso correspondiente.
    *
    * @param lista    lista de coches a revisar
    * @param indexMec índice del mecánico asignado (módulo nº mecánicos)
    */
    private void resolverAveriasConMecanico(ArrayList<? extends Coche> lista, int indexMec)
    {
        if (mecanicos == null || mecanicos.length == 0) {
            return;
        }
        Mecanico mec = mecanicos[indexMec % mecanicos.length];

        for (Coche c : lista) {
            if (c.isAveriado()) {
                if (c.getTiempoReparacion() == -1) {
                    c.setTiempoReparacion(mec.getTiempoReparacion());
                }

                c.setTiempoReparacion(c.getTiempoReparacion() - 1);

                if (c.getTiempoReparacion() <= 0) {
                    if (gestorPlanta != null) {
                        gestorPlanta.llamarMecanico(mec, c);
                    } else {
                        mec.repararCoche(c);
                    }
                    String perfil = mec.esEficiente() ? "eficiente" : "estándar";
                    String msg = mec.getNombre() + " " + mec.getApellidos() + " (" + perfil + ") ha
completado la reparación (" + mec.getReparacionesRealizadas() + " reparaciones totales).";
                    cadenaMontaje.notifyObservadores(msg);
                }
            }
        }
    }

    /**
    * @return {@code true} si las 3 cadenas tienen todos sus coches en
    * estado TERMINADO
    */
    private boolean comprobarFinMontaje() {
        return todosTerminados(cadenaMontaje.getCadenaBiplaza())
            && todosTerminados(cadenaMontaje.getCadenaTurismo())
            && todosTerminados(cadenaMontaje.getCadenaFurgoneta());
    }

    /**
    * @param lista lista de coches a evaluar

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

* @return {@code true} si la lista está vacía o todos sus coches están
*   en estado TERMINADO
*/
private boolean todosTerminados(ArrayList<? extends Coche> lista) {
    if (lista.isEmpty()) {
        return true;
    }
    for (Coche c : lista) {
        if (c.getEstadoMontaje() != EstadoMontaje.TERMINADO) {
            return false;
        }
    }
    return true;
}

/**
 * Imprime por consola un resumen final del trabajo realizado por
 * mecánicos y administrador, adaptado al nivel de simulación.
 */
private void imprimirResumenTrabajadores() {
    notifyObservadores("RESUMEN");

    if (tipoSimulacion >= 2 && mecanicos != null) {
        for (Mecanico m : mecanicos) {
            System.out.println("- El mecánico " + m.getNombre() + " " + m.getApellidos() + " ha
hecho " + m.getReparacionesRealizadas() + " reparaciones.");
        }
    }
    if (tipoSimulacion == 3 && admin != null) {
        System.out.println("- El administrador " + admin.getNombre() + " " +
admin.getApellidos() + " ha restaurado la luz: " + admin.getRestauracionesRealizadas() + "
veces.");
    }
    if (tipoSimulacion == 1) {
        System.out.println("- Simulación simple completada sin incidencias.");
    }
}

/** {@inheritDoc} */
@Override
public void notifyObservadores(String message) {
    for (Observador observador : observadores) {
        observador.update(message);
    }
}

/** {@inheritDoc} */
@Override

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```
public void addObserver(Observador observador) {
    observadores.add(observador);
}

/** {@inheritDoc} */
@Override
public void removeObservador(Observador observador) {
    observadores.remove(observador);
}

/** @return cadena de montaje asociada */
public CadenaMontaje getCadenaMontaje() {
    return cadenaMontaje;
}

/** @param cadenaMontaje nueva cadena de montaje */
public void setCadenaMontaje(CadenaMontaje cadenaMontaje) {
    this.cadenaMontaje = cadenaMontaje;
}

/** @return sistema de gestión asociado */
public SistemaGestion getSistemaGestion() {
    return sistemaGestion;
}

/** @param sistemaGestion nuevo sistema de gestión */
public void setSistemaGestion(SistemaGestion sistemaGestion) {
    this.sistemaGestion = sistemaGestion;
}

/** @return nivel de simulación (1, 2 ó 3) */
public int getTipoSimulacion() {
    return tipoSimulacion;
}

/** @param tipoSimulacion nuevo nivel de simulación */
public void setTipoSimulacion(int tipoSimulacion) {
    this.tipoSimulacion = tipoSimulacion;
}

/** @return {@code true} si actualmente hay apagón */
public boolean isCaidaDeLuz() {
    return caidaDeLuz;
}

/** @param caidaDeLuz nuevo estado de apagón */
public void setCaidaDeLuz(boolean caidaDeLuz) {
    this.caidaDeLuz = caidaDeLuz;
}
```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

    }

    /** @return {@code true} si el sistema de gestión está bloqueado */
    public boolean isSistemaGestionBloqueado() {
        return sistemaGestionBloqueado;
    }

    /** @param sistemaGestionBloqueado nuevo estado de bloqueo */
    public void setSistemaGestionBloqueado(boolean sistemaGestionBloqueado) {
        this.sistemaGestionBloqueado = sistemaGestionBloqueado;
    }
}

```

7.10 factory_main.java

```

import com.practica.fabrica.*;
import com.practica.montaje.*;
import com.practica.dashboard.*;
import com.practica.planificador.*;
import com.practica.personal.*;
import com.practica.motor.*;
import com.practica.tapiceria.*;
import com.practica.rueda.*;
import com.practica.vehiculo.*;

import java.time.LocalDate;
import java.text.SimpleDateFormat;
import java.util.Date;
import java.util.List;
import java.util.Map;
import java.util.Scanner;

/**
 * Clase principal de la aplicación de gestión de la fábrica de vehículos.
 * <p>
 * Expone un menú textual por consola que permite al usuario acceder a las
 * funciones de la fábrica: gestión de almacén (componentes y vehículos en
 * cadena), gestión de trabajadores, consulta de stock, búsqueda de
 * empleados, simulación del montaje y listados/estadísticas. * <p>
 * Coordina las instancias singleton de {@link SistemaGestion},
 * {@link CadenaMontaje} y {@link Dashboard} que se comparten entre todas
 * las opciones del menú. *
 * @author Shao Capilla Sanz
 */
public class factory_main {

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

/** Fachada del sistema de gestión sobre el almacén de datos. */
public static SistemaGestion sistemaGestion;
/** Cadena de montaje activa con los vehículos pendientes. */
public static CadenaMontaje cadenaMontaje;
/** Dashboard observador que muestra los eventos en consola. */
public static Dashboard dashboard;
/** Scanner único para leer entrada del usuario. */
public static Scanner sc = new Scanner(System.in);

/**
 * Punto de entrada del programa. Inicializa el almacén, el sistema de
 * gestión, la cadena de montaje y el dashboard, conecta el dashboard
 * como observador del almacén y de la cadena, y entra en el bucle del
 * menú principal hasta que el usuario elige salir.
 *
 * @param args argumentos de línea de comandos (no se utilizan)
 */
public static void main(String[] args) {

    AlmacenDatos almacen = new AlmacenDatos();
    sistemaGestion = new SistemaGestion(almacen);
    cadenaMontaje = new CadenaMontaje();
    dashboard = new Dashboard(new VisualizarConsola());

    almacen.addObserver(dashboard);
    cadenaMontaje.addObserver(dashboard);

    int opcion = -1;
    while (opcion != 0) {
        System.out.println("\nGESTIÓN FÁBRICA DE VEHÍCULOS");
        System.out.println("1. Gestión de Almacén");
        System.out.println("2. Gestión de Trabajadores");
        System.out.println("3. Consultar stock");
        System.out.println("4. Buscar empleados");
        System.out.println("5. Iniciar Simulación");
        System.out.println("6. Listados y estadísticas");
        System.out.println("7. Reiniciar Sistema");
        System.out.println("0. Salir");
        System.out.print("Elige una opción: ");
        opcion = sc.nextInt();

        switch (opcion) {
            case 1:
                menuAlmacen();
                break;
            case 2:
                menuTrabajadores();
                break;

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

        case 3:
            mostrarStock();
            break;
        case 4:
            menuBusqueda();
            break;
        case 5:
            menuSimulacion();
            break;
        case 6:
            menuListados();
            break;
        case 7:
            sistemaGestion.resetSistema();
            dashboard.update("Sistema reiniciado correctamente.");
            break;
        case 0:
            System.out.println("Saliendo...");
            break;
        default:
            System.out.println("Opción no válida.");
    }
}

/**
 * Comprueba que hay stock suficiente para fabricar un vehículo más,
 * teniendo en cuenta los vehículos ya pendientes en la cadena. Si no
 * hay stock muestra un mensaje y devuelve {@code false}.
 *
 * @return {@code true} si se puede añadir un vehículo más;
 *         {@code false} en caso contrario
 */
private static boolean comprobarStockParaNuevoVehiculo() {
    cadenaMontaje.purgarTerminados();
    int yaEnCadena = cadenaMontaje.getCadenaBiplaza().size() +
        cadenaMontaje.getCadenaTurismo().size() + cadenaMontaje.getCadenaFurgoneta().size();
    ComprobacionStock res = sistemaGestion.comprobarStock(yaEnCadena + 1);
    if (res != ComprobacionStock.OK) {
        dashboard.mostrarError("No se añade el vehículo, stock insuficiente de " + res);
        return false;
    }
    return true;
}

/**
 * Delega la creación de un vehículo en el gestor de planta si existe;
 * en caso contrario añade el vehículo directamente a la cadena.

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

*
* @param tipo    tipo de vehículo ("Biplaza", "Turismo" o "Furgoneta")
* @param color   color del vehículo
* @param plazas  número de plazas
* @param pesoMax peso máximo autorizado en kg
* @param tara    tara del vehículo en kg
*/
private static void delegarAGestor(String tipo, String color, int plazas,
    double pesoMax, double tara) {
    List<GestorPlanta> gestores = sistemaGestion.consultarGestorPlanta();
    if (!gestores.isEmpty()) {
        GestorPlanta g = gestores.get(0);
        switch (tipo) {
            case "Biplaza":
                g.configurarBiplazas(cadenaMontaje, 1, color, tara, pesoMax, null, null, null);
                cadenaMontaje.getCadenaBiplaza()
                    .get(cadenaMontaje.getCadenaBiplaza().size() - 1)
                    .setPlazas(plazas);
                break;
            case "Turismo":
                g.configurarTurismos(cadenaMontaje, 1, color, tara, pesoMax, null, null, null);
                cadenaMontaje.getCadenaTurismo()
                    .get(cadenaMontaje.getCadenaTurismo().size() - 1)
                    .setPlazas(plazas);
                break;
            case "Furgoneta":
                g.configurarFurgonetas(cadenaMontaje, 1, color, tara, pesoMax, null, null, null);
                cadenaMontaje.getCadenaFurgoneta()
                    .get(cadenaMontaje.getCadenaFurgoneta().size() - 1)
                    .setPlazas(plazas);
                break;
        }
    } else {
        switch (tipo) {
            case "Biplaza":
                cadenaMontaje.agregarBiplaza(new BiplazaDeportivo(color, plazas, pesoMax,
tara, null, null, null));
                dashboard.mostrarConfirmacion("Biplaza Deportivo añadido a la cadena.");
                break;
            case "Turismo":
                cadenaMontaje.agregarTurismo(new Turismo(color, plazas, pesoMax, tara, null,
null, null));
                dashboard.mostrarConfirmacion("Turismo añadido a la cadena.");
                break;
            case "Furgoneta":
                cadenaMontaje.agregarFurgoneta(new Furgoneta(color, plazas, pesoMax, tara,
null, null, null));

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

        dashboard.mostrarConfirmacion("Furgoneta añadida a la cadena.");
        break;
    }
}
}

/**
 * Solicita al usuario un componente del almacén (motor, tapicería o
 * rueda) y permite modificar sus atributos. Muestra la lista numerada
 * de los elementos disponibles para que el usuario elija por índice.
 */
private static void actualizarComponenteAlmacen() {
    System.out.println("Actualizar componente:");
    System.out.println(" 1) Motor  2) Tapicería  3) Rueda");
    System.out.print("Tipo: ");
    int tipo = sc.nextInt();
    List<?> lista;
    switch (tipo) {
        case 1:
            lista = sistemaGestion.consultarMotor();
            break;
        case 2:
            lista = sistemaGestion.consultarTapiceria();
            break;
        case 3:
            lista = sistemaGestion.consultarRueda();
            break;
        default:
            System.out.println("Tipo no válido.");
            return;
    }

    if (lista.isEmpty()) {
        dashboard.update("No hay elementos para actualizar.");
        return;
    }

    for (int i = 0; i < lista.size(); i++) {
        System.out.println(" [" + i + "] " + lista.get(i));
    }

    System.out.print("Índice a actualizar: ");
    int indice = sc.nextInt();

    if (indice < 0 || indice >= lista.size()) {
        System.out.println("Índice fuera de rango.");
        return;
    }
}

```


Practica de Programación Orientada a Objeto – Curs 2025-2026

```

switch (tipo) {
    case 1: {
        Motor m = (Motor) lista.get(indice);
        System.out.print("Nueva cilindrada: ");
        m.setCilindrada(sc.nextDouble());
        System.out.print("Nueva potencia: ");
        m.setPotencia(sc.nextInt());
        System.out.print("Nuevo número cilindros: ");
        m.setNumeroCilindros(sc.nextInt());
        dashboard.mostrarConfirmacion("Motor actualizado.");
        break;
    }
    case 2: {
        Tapiceria t = (Tapiceria) lista.get(indice);
        sc.nextLine();
        System.out.print("Nuevo color: ");
        t.setColor(sc.nextLine());
        System.out.print("Nuevos m²: ");
        t.setMetrosCuadrados(sc.nextDouble());
        dashboard.mostrarConfirmacion("Tapicería actualizada.");
        break;
    }
    case 3: {
        Rueda r = (Rueda) lista.get(indice);
        System.out.print("Nuevo ancho: ");
        r.setAncho(sc.nextInt());
        System.out.print("Nueva pulgadas llanta: ");
        r.setPulgadasLlanta(sc.nextInt());
        System.out.print("Nuevo índice carga: ");
        r.setIndiceCarga(sc.nextInt());
        System.out.print("Nuevo código velocidad: ");
        r.setVelocidad(sc.nextInt());
        dashboard.mostrarConfirmacion("Rueda actualizada.");
        break;
    }
}
}

/**
 * Submenú de gestión de almacén. Permite registrar componentes
 * (motores Gasolina/Eléctrico/Híbrido, tapicerías Tela/Cuero/Alcántara,
 * ruedas Normal/Deportiva/Todoterreno), añadir vehículos a la cadena
 * (Biplaza, Turismo, Furgoneta) y actualizar componentes existentes.
 * Las ruedas se registran siempre en juegos de 4.
 */
public static void menuAlmacen() {
    int op = -1;

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

while (op != 0) {
    System.out.println("\nMENÚ GESTIÓN DE ALMACÉN");
    System.out.println("1. Añadir Motor Gasolina");
    System.out.println("2. Añadir Motor Eléctrico");
    System.out.println("3. Añadir Motor Híbrido");
    System.out.println("4. Añadir Tapicería Tela");
    System.out.println("5. Añadir Tapicería Cuero");
    System.out.println("6. Añadir Tapicería Alcántara");
    System.out.println("7. Añadir Rueda Normal");
    System.out.println("8. Añadir Rueda Deportiva");
    System.out.println("9. Añadir Rueda Todoterreno");
    System.out.println("-- VEHÍCULOS A ENSAMBLAR --");
    System.out.println("10. Añadir Biplaza Deportivo a la cadena");
    System.out.println("11. Añadir Turismo a la cadena");
    System.out.println("12. Añadir Furgoneta a la cadena");
    System.out.println("13. Actualizar componente del almacén");
    System.out.println("0. Volver al menú principal");
    System.out.print("Opción: ");
    op = sc.nextInt();
    switch (op) {
        case 1:
            System.out.print("Cilindrada: ");
            double cGasolina = sc.nextDouble();
            System.out.print("Potencia: ");
            int pGasolina = sc.nextInt();
            System.out.print("Cilindros: ");
            int numCilGasolina = sc.nextInt();
            sistemaGestion.registrarMotor(new Gasolina(cGasolina, pGasolina,
numCilGasolina));
            dashboard.mostrarConfirmacion("Motor Gasolina registrado.");
            break;
        case 2:
            System.out.print("Cilindrada: ");
            double cElectrico = sc.nextDouble();
            System.out.print("Potencia: ");
            int pcElectrico = sc.nextInt();
            System.out.print("Cilindros: ");
            int numCilcElectrico = sc.nextInt();
            sistemaGestion.registrarMotor(new Electrico(cElectrico, pcElectrico,
numCilcElectrico));
            dashboard.mostrarConfirmacion("Motor Electrico registrado.");
            break;
        case 3:
            System.out.print("Cilindrada: ");
            double cHibrido = sc.nextDouble();
            System.out.print("Potencia: ");
            int pcHibrido = sc.nextInt();
            System.out.print("Cilindros: ");

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

        int numCilcHibrido = sc.nextInt();
        sistemaGestion.registrarMotor(new Hibrido(cHibrido, pcHibrido,
numCilcHibrido));
        dashboard.mostrarConfirmacion("Motor Hibrido registrado.");
        break;
    case 4:
        System.out.print("Color: ");
        sc.nextLine();
        String cTela = sc.nextLine();
        System.out.print("Metros Cuadrados: ");
        double mcTela = sc.nextDouble();
        sistemaGestion.registrarTapiceria(new Tela(cTela, mcTela));
        dashboard.mostrarConfirmacion("Tapiceria Tela registrado.");
        break;
    case 5:
        System.out.print("Color: ");
        sc.nextLine();
        String cCuero = sc.nextLine();
        System.out.print("Metros Cuadrados: ");
        double mcCuero = sc.nextDouble();
        sistemaGestion.registrarTapiceria(new Cuero(cCuero, mcCuero));
        dashboard.mostrarConfirmacion("Tapiceria Cuero registrado.");
        break;
    case 6:
        System.out.print("Color: ");
        sc.nextLine();
        String cAlcantara = sc.nextLine();
        System.out.print("Metros Cuadrados: ");
        double mcAlcantara = sc.nextDouble();
        sistemaGestion.registrarTapiceria(new Alcantara(cAlcantara, mcAlcantara));
        dashboard.mostrarConfirmacion("Tapiceria Alcantara registrado.");
        break;
    case 7:
        System.out.print("Ancho: ");
        int aNormal = sc.nextInt();
        System.out.print("Pulgadas Llanta: ");
        int pLlantaNormal = sc.nextInt();
        System.out.print("Indice Carga: ");
        int indiCargaNormal = sc.nextInt();
        System.out.print("Codigo Velocial: ");
        int codVelNormal = sc.nextInt();
        for (int i = 0; i < 4; i++) {
            sistemaGestion.registrarRueda(new Normal(aNormal, pLlantaNormal,
indiCargaNormal,codVelNormal));
        }
        dashboard.mostrarConfirmacion("4 Ruedas Normales registradas.");
        break;
    case 8:

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

        System.out.print("Ancho: ");
        int aDeportivo = sc.nextInt();
        System.out.print("Pulgadas Llanta: ");
        int pLlantaDeportivo = sc.nextInt();
        System.out.print("Indice Carga: ");
        int indiCargaDeportivo = sc.nextInt();
        System.out.print("Codigo Velocial: ");
        int codVelDeportivo = sc.nextInt();
        for (int i = 0; i < 4; i++) {
            sistemaGestion.registrarRueda(new Deportivo(aDeportivo, pLlantaDeportivo,
            indiCargaDeportivo, codVelDeportivo));
        }
        dashboard.mostrarConfirmacion("4 Ruedas Deportivas registradas.");
        break;
    case 9:
        System.out.print("Ancho: ");
        int aTodoterreno = sc.nextInt();
        System.out.print("Pulgadas Llanta: ");
        int pLlantaTodoterreno = sc.nextInt();
        System.out.print("Indice Carga: ");
        int indiCargaTodoterreno = sc.nextInt();
        System.out.print("Codigo Velocial: ");
        int codVelTodoterreno = sc.nextInt();
        for (int i = 0; i < 4; i++) {
            sistemaGestion.registrarRueda(new Todoterreno(aTodoterreno,
            pLlantaTodoterreno, indiCargaTodoterreno, codVelTodoterreno));
        }
        dashboard.mostrarConfirmacion("4 Ruedas Todoterrenos registradas.");
        break;
    case 10:
        System.out.print("Color: ");
        sc.nextLine();
        String colorBiplaza = sc.nextLine();
        System.out.print("Nº plazas: ");
        int plazasBiplaza = sc.nextInt();
        System.out.print("Peso Autorizado: ");
        double pesoAutorBiplaza = sc.nextDouble();
        System.out.print("Tara Vehiculo: ");
        double taraVehiBiplaza = sc.nextDouble();
        if (!comprobarStockParaNuevoVehiculo()) break;
        delegarAGestor("Biplaza", colorBiplaza, plazasBiplaza, pesoAutorBiplaza,
        taraVehiBiplaza);
        break;
    case 11:
        System.out.print("Color: ");
        sc.nextLine();
        String colorTurismo = sc.nextLine();
        System.out.print("Nº plazas: ");

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

        int plazasTurismo = sc.nextInt();
        System.out.print("Peso Autorizado: ");
        double pesoAutorTurismo = sc.nextDouble();
        System.out.print("Tara Vehiculo: ");
        double taraVehiTurismo = sc.nextDouble();
        if (!comprobarStockParaNuevoVehiculo()) break;
        delegarAGestor("Turismo", colorTurismo, plazasTurismo, pesoAutorTurismo,
taraVehiTurismo);
        break;
    case 12:
        System.out.print("Color: ");
        sc.nextLine();
        String colorFurgoneta = sc.nextLine();
        System.out.print("Nº plazas: ");
        int plazasFurgoneta = sc.nextInt();
        System.out.print("Peso Autorizado: ");
        double pesoAutorFurgoneta = sc.nextDouble();
        System.out.print("Tara Vehiculo: ");
        double taraVehiFurgoneta = sc.nextDouble();
        if (!comprobarStockParaNuevoVehiculo()) break;
        delegarAGestor("Furgoneta", colorFurgoneta, plazasFurgoneta,
pesoAutorFurgoneta, taraVehiFurgoneta);
        break;
    case 13:
        actualizarComponenteAlmacen();
        break;
    case 0:
        System.out.println("Volviendo al menú principal...");
        break;
    default:
        System.out.println("Opción no válida.");
    }
}
}

/**
 * Submenú de gestión de trabajadores. Permite dar de alta operarios,
 * mecánicos, gestores de planta y administradores de sistema. Todos
 * comparten los mismos datos básicos (nombre, apellidos, DNI, dirección,
 * NSS y salario) con fecha de ingreso igual a la actual.
 */
public static void menuTrabajadores() {
    int op = -1;
    while (op != 0) {
        System.out.println("\nMENÚ GESTIÓN DE TRABAJADORES");
        System.out.println("1. Añadir Operario");
        System.out.println("2. Añadir Mecánico");
        System.out.println("3. Añadir Gestor de Planta");
    }
}

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

System.out.println("4. Añadir Administrador de Sistema");
System.out.println("0. Volver al menú principal");
System.out.print("Opción: ");
op = sc.nextInt();
switch (op) {
    case 0:
        System.out.println("Volviendo al menú principal...");
        break;
    case 1: case 2: case 3: case 4:
        sc.nextLine();
        System.out.print("Nombre: ");
        String nombre = sc.nextLine();
        System.out.print("Apellidos: ");
        String apellidos = sc.nextLine();
        System.out.print("DNI: ");
        String dni = sc.nextLine();
        System.out.print("Dirección: ");
        String dir = sc.nextLine();
        System.out.print("Nº Seg. Social: ");
        String segSocial = sc.nextLine();
        System.out.print("Salario: ");
        double salario = sc.nextDouble();
        switch (op) {
            case 1:
                sistemaGestion.registrarOperario(
                    new Operario(nombre, apellidos, dni, dir, segSocial, salario,
LocalDate.now()));
                dashboard.mostrarConfirmacion("Operario registrado.");
                break;
            case 2:
                sistemaGestion.registrarMecanico(
                    new Mecanico(nombre, apellidos, dni, dir, segSocial, salario,
LocalDate.now(),dashboard));
                dashboard.mostrarConfirmacion("Mecánico registrado.");
                break;
            case 3:
                sistemaGestion.registrarGestorPlanta(
                    new GestorPlanta(nombre, apellidos, dni, dir, segSocial, salario,
LocalDate.now(),dashboard));
                dashboard.mostrarConfirmacion("Gestor de Planta registrado.");
                break;
            case 4:
                sistemaGestion.registrarAdministradorSistema(
                    new AdministradorSistema(nombre, apellidos, dni, dir, segSocial, salario,
LocalDate.now()));
                dashboard.mostrarConfirmacion("Administrador de Sistema registrado.");
                break;
        }
}

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

        break;
    default:
        System.out.println("Opción no válida.");
    }
}
}

/**
 * Submenú de búsqueda de empleados. Permite buscar operarios por
 * nombre, listar operarios eficientes (>10 montajes), buscar a un
 * trabajador por DNI en cualquier perfil y listar todos los operarios
 * registrados con sus métricas.
 */
public static void menuBusqueda() {
    int op = -1;
    while (op != 0) {
        System.out.println("\nMENÚ BUSCAR EMPLEADOS");
        System.out.println("1. Buscar operario por nombre");
        System.out.println("2. Listar operarios eficientes");
        System.out.println("3. Buscar trabajador por DNI");
        System.out.println("4. Listar todos los operarios");
        System.out.println("5. Buscar mecánico por nombre");
        System.out.println("6. Listar todos los mecánicos");
        System.out.println("7. Listar todos los gestores de planta");
        System.out.println("8. Listar todos los administradores de sistema");
        System.out.println("0. Volver al menú principal");
        System.out.print("Opción: ");
        op = sc.nextInt(); sc.nextLine();
        switch (op) {
            case 1:
                System.out.print("Nombre a buscar: ");
                String nombre = sc.nextLine();
                List<Operario> encontrados =
sistemaGestion.buscarOperariosPorNombre(nombre);
                if (encontrados.isEmpty()) {
                    dashboard.update("No se encontraron operarios con ese nombre.");
                } else {
                    dashboard.update("Operarios Encontrados: ");
                    for (Operario o : sistemaGestion.buscarOperariosPorNombre(nombre)) {
                        System.out.println(" - " + o.getNombre() + " " + o.getApellidos());
                    }
                }
                break;
            case 2:
                List<Operario> eficientes = sistemaGestion.buscarOperariosEficientes();
                if(eficientes.isEmpty()) {
                    dashboard.update("No hay operarios eficientes registrados.");
                } else {

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

        dashboard.update("Listando Operarios Eficientes: ");
        for (Operario o : sistemaGestion.buscarOperariosEficientes()) {
            System.out.println(" - " + o.getNombre() + " (montajes: " +
o.getMontajesRealizados() + ")");
        }
    }

    break;
case 3:
    System.out.print("DNI: ");
    String dni = sc.nextLine();
    String trabajadorDni = sistemaGestion.buscarTrabajadorPorDni(dni);
    if (trabajadorDni == null) {
        dashboard.update("No se encontró ningún trabajador con ese DNI.");
    } else {
        dashboard.update("Trabajador encontrado, " + trabajadorDni);
    }
    break;
case 4:
    List<Operario> todosOperarios = sistemaGestion.consultarOperario();
    if (todosOperarios.isEmpty()) {
        dashboard.update("No hay operarios registrados.");
    } else {
        dashboard.update("Listando Operarios: ");
        for (Operario o : todosOperarios) {
            String perfil = o.esEficiente() ? "eficiente" : "estándar";
            System.out.println(" - " + o.getNombre() + " " + o.getApellidos() + " | DNI: "
+ o.getDni() + " | montajes: " + o.getMontajesRealizados() + " (" + perfil + ")");
        }
    }
    break;
case 5:
    System.out.print("Nombre del mecánico a buscar: ");
    String nombreMecanico = sc.nextLine();
    List<Mecanico> mecanicos =
sistemaGestion.buscarMecanicosPorNombre(nombreMecanico);
    if (mecanicos.isEmpty()) {
        dashboard.update("No se encontraron mecánicos con ese nombre.");
    } else {
        dashboard.update("Listando Mecánicos: ");
        for (Mecanico m : mecanicos) {
            System.out.println(" - " + m.getNombre() + " " + m.getApellidos());
        }
    }
    break;
case 6:
    List<Mecanico> todosMecanicos = sistemaGestion.consultarMecanico();
    if (todosMecanicos.isEmpty()) {

```


Practica de Programación Orientada a Objeto – Curs 2025-2026

```

        dashboard.update("No hay mecánicos registrados.");
    } else {
        dashboard.update("Listando Mecánicos: ");
        for (Mecanico m : todosMecanicos) {
            String perfil = m.esEficiente() ? "eficiente" : "estándar";
            System.out.println(" - " + m.getNombre() + " " + m.getApellidos() + " | DNI: "
+ m.getDni() + " | reparaciones: " + m.getReparacionesRealizadas() + " (" + perfil + ")");
        }
    }
    break;
case 7:
    List<GestorPlanta> todosGestor = sistemaGestion.consultarGestorPlanta();
    if (todosGestor.isEmpty()) {
        dashboard.update("No hay operarios registrados.");
    } else {
        dashboard.update("Listando Operarios: ");
        for (GestorPlanta g : todosGestor) {
            System.out.println(" - " + g.getNombre() + " " + g.getApellidos() + " | DNI: " +
g.getDni());
        }
    }
    break;
case 8:
    List<AdministradorSistema> todosAdmin =
sistemaGestion.consultarAdministradorSistema();
    if (todosAdmin.isEmpty()) {
        dashboard.update("No hay operarios registrados.");
    } else {
        dashboard.update("Listando Operarios: ");
        for (AdministradorSistema a : todosAdmin) {
            System.out.println(" - " + a.getNombre() + " " + a.getApellidos() + " | DNI: " +
a.getDni());
        }
    }
    break;
case 0:
    System.out.println("Volviendo al menú principal...");
    break;
default:
    System.out.println("Opción no válida.");
}
}
}

/**
 * Muestra por consola el stock actual del almacén (motores, tapicerías
 * y ruedas) y el número de vehículos pendientes en la cadena y ya
 * ensamblados, separados por tipo.

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

*/
public static void mostrarStock() {
    dashboard.update("STOCK DEL ALMACÉN");
    System.out.println("Motores disponibles: " + sistemaGestion.consultarMotor().size());
    System.out.println("Tapicerías disponibles: " +
sistemaGestion.consultarTapiceria().size());
    System.out.println("Ruedas disponibles: " + sistemaGestion.consultarRueda().size());

    int pendBip = contarPendientes(cadenaMontaje.getCadenaBiplaza());
    int pendTur = contarPendientes(cadenaMontaje.getCadenaTurismo());
    int pendFur = contarPendientes(cadenaMontaje.getCadenaFurgoneta());
    int ensBip = sistemaGestion.consultarBiplazasDeportivos().size();
    int ensTur = sistemaGestion.consultarTurismo().size();
    int ensFur = sistemaGestion.consultarFurgoneta().size();

    System.out.println("\nVEHÍCULOS EN CADENA (pendientes de ensamblar)");
    System.out.println(" Biplazas: " + pendBip);
    System.out.println(" Turismos: " + pendTur);
    System.out.println(" Furgonetas: " + pendFur);
    System.out.println("VEHÍCULOS ENSAMBLADOS (histórico)");
    System.out.println(" Biplazas: " + ensBip);
    System.out.println(" Turismos: " + ensTur);
    System.out.println(" Furgonetas: " + ensFur);
    System.out.println(" TOTAL coches: " + sistemaGestion.consultarTotalCoches());
}

/**
 * Cuenta los coches de la lista que aún no han alcanzado el estado
 * TERMINADO.
 *
 * @param lista lista de coches a evaluar
 * @return número de coches pendientes
 */
private static int contarPendientes(List<? extends Coche> lista) {
    int cont = 0;
    for (Coche c : lista) {
        if (c.getEstadoMontaje() != EstadoMontaje.TERMINADO) {
            cont++;
        }
    }
    return cont;
}

/**
 * Submenú de simulación. Permite elegir entre simulación simple,
 * compleja (con averías y mecánicos) o muy compleja (averías + apagones
 * con administrador). Antes de iniciar la simulación valida los
 * requisitos: vehículos en cadena, stock suficiente, presencia de

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

* mecánicos y/o administrador según el nivel.
*/
public static void menuSimulacion() {
    int op = -1;
    while (op != 0) {
        System.out.println("\nMENÚ INICIAR SIMULACIÓN");
        System.out.println("1. Simple (sin averías)");
        System.out.println("2. Compleja (con mecánicos)");
        System.out.println("3. Muy Compleja (mecánicos + apagones)");
        System.out.println("0. Volver al menú principal");
        System.out.print("Tipo de simulación: ");
        op = sc.nextInt();

        if (op == 0) {
            System.out.println("Volviendo al menú principal...");
            break;
        }

        if (op < 1 || op > 3) {
            dashboard.mostrarError("Opción no válida.");
            continue;
        }

        cadenaMontaje.purgarTerminados();
        int totalCoches = cadenaMontaje.getCadenaBiplaza().size() +
            cadenaMontaje.getCadenaTurismo().size() + cadenaMontaje.getCadenaFurgoneta().size();
        if (totalCoches == 0) {
            dashboard.mostrarError("No hay vehículos en la cadena. Añade vehículos desde
            Gestión de Almacén (opciones 10-12).");
            break;
        }

        ComprobacionStock res = sistemaGestion.comprobarStock(totalCoches);
        if (res != ComprobacionStock.OK) {
            dashboard.mostrarError("Simulación cancelada, stock insuficiente de " + res);
            break;
        }

        if (op >= 2 && sistemaGestion.consultarMecanico().isEmpty()) {
            dashboard.mostrarError("Simulación cancelada, se requiere al menos un
            mecánico.");
            break;
        }

        if (op == 2) {
            boolean hayEficiente = false;
            boolean hayEstandar = false;
            for (Mecanico m : sistemaGestion.consultarMecanico()) {
                if (m.esEficiente()) {

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

        hayEficiente = true;
    } else {
        hayEstandar = true;
    }
}
if (!hayEficiente && hayEstandar) {
    dashboard.update("[AVISO] Aún no hay mecánicos eficientes (>20 reparaciones).
" + "La simulación continuará sólo con mecánicos estándar.");
}
}
if (op == 3 && sistemaGestion.consultarAdministradorSistema().isEmpty()) {
    dashboard.mostrarError("Simulación cancelada, se requiere un Administrador de
Sistema para gestionar apagones.");
    break;
}

Mecanico[] mecs = (op == 1) ? new Mecanico[0] :
sistemaGestion.consultarMecanico().toArray(new Mecanico[0]);

AdministradorSistema admin = (op == 3 &&
!sistemaGestion.consultarAdministradorSistema().isEmpty()) ?
sistemaGestion.consultarAdministradorSistema().get(0) : null;

dashboard.update("Iniciando simulación con " + totalCoches + " vehículos...");

Planificador planificador = new Planificador(cadenaMontaje, sistemaGestion, op,
mecs, admin, dashboard);
planificador.addObservador(dashboard);

planificador.iniciarSimulacion();
break;
}
}

/**
 * Submenú de listados y estadísticas. Permite obtener listados
 * ordenados de operarios, filtros sobre vehículos terminados por
 * motor o tapicería, ranking de configuraciones más ensambladas y la
 * consulta del historial de operaciones por fecha.
 */
public static void menuListados() {
    int op = -1;
    while (op != 0) {
        System.out.println("\nMENÚ LISTADOS Y ESTADÍSTICAS");
        System.out.println("1. Operarios ordenados (nombre, apellidos)");
        System.out.println("2. Operarios por productividad (mínimo de montajes)");
        System.out.println("3. Vehículos terminados");
        System.out.println("4. Vehículos terminados por tipo de motor");
    }
}

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

System.out.println("5. Vehículos terminados por tipo de tapicería");
System.out.println("6. Vehículos terminados ordenados por tipo");
System.out.println("7. Configuraciones más ensambladas");
System.out.println("8. Consultar historial por fecha");
System.out.println("0. Volver al menú principal");
System.out.print("Opción: ");
op = sc.nextInt();
switch (op) {
    case 1: {
        List<Operario> lista = sistemaGestion.ordenarOperarios();
        if (lista.isEmpty()) {
            dashboard.update("No hay operarios registrados.");
        } else {
            dashboard.update("Listando Operarios Ordenas:");
            for (Operario o : lista) {
                System.out.println(" - " + o.getNombre() + " " + o.getApellidos() + "
(montajes: " + o.getMontajesRealizados() + ")");
            }
        }
        break;
    }
    case 2: {
        System.out.print("Mínimo de montajes: ");
        int min = sc.nextInt();
        List<Operario> lista = sistemaGestion.obtenerOperariosProductividad(min);
        if (lista.isEmpty()) {
            dashboard.update("Ningún operario alcanza ese umbral.");
        } else {
            dashboard.update("Listando Operarios: ");
            for (Operario o : lista) {
                System.out.println(" - " + o.getNombre() + " " + o.getApellidos() + "
(montajes: " + o.getMontajesRealizados() + ")");
            }
        }
        break;
    }
    case 3: {
        List<Coche> coches =
sistemaGestion.obtenerCochesTerminados(cadenaMontaje);
        if (coches.isEmpty()) {
            dashboard.update("No hay vehículos terminados todavía.");
        } else {
            dashboard.update("Listando vehículos terminados: ");
            for (Coche c : coches) {
                System.out.println(" - " + c.tipoCoche() + " (" + c.getColor() + ")");
            }
        }
        break;
    }
}

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

    }
    case 4: {
        sc.nextLine();
        System.out.print("Tipo de motor (Gasolina/Electrico/Hibrido): ");
        String tipo = sc.nextLine();
        List<Coche> terminados =
sistemaGestion.obtenerCochesTerminados(cadenaMontaje);
        List<Coche> filtrados = sistemaGestion.filtrarTipoMotor(terminados, tipo);
        if (filtrados.isEmpty()) {
            dashboard.update("Sin coincidencias.");
        } else {
            dashboard.update("Listado vehículos con motor " + tipo);
            for (Coche c : filtrados) {
                System.out.println(" - " + c.tipoCoche() + " (" + c.getColor() + ")");
            }
        }
        break;
    }
    case 5: {
        sc.nextLine();
        System.out.print("Tipo de tapicería (Tela/Cuero/Alcantara): ");
        String tipo = sc.nextLine();
        List<Coche> terminados =
sistemaGestion.obtenerCochesTerminados(cadenaMontaje);
        List<Coche> filtrados = sistemaGestion.filtrarTipoTapiceria(terminados, tipo);
        if (filtrados.isEmpty()) {
            dashboard.update("Sin coincidencias.");
        } else {
            dashboard.update("Listando vehículos con tapicería " + tipo);
            for (Coche c : filtrados) {
                System.out.println(" - " + c.tipoCoche() + " (" + c.getColor() + ")");
            }
        }
        break;
    }
    case 6: {
        List<Coche> terminados =
sistemaGestion.obtenerCochesTerminados(cadenaMontaje);
        List<Coche> ordenados = sistemaGestion.obtenerCochesOrdenados(terminados);
        if (ordenados.isEmpty()) {
            dashboard.update("No hay vehículos terminados.");
        } else {
            dashboard.update("Listando vehículos terminados ordenados por tipo: ");
            for (Coche c : ordenados) {
                System.out.println(" - " + c.tipoCoche() + " (" + c.getColor() + ")");
            }
        }
        break;
    }

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```

    }
    case 7: {
        Map<String, Integer> conf =
sistemaGestion.getConfiguracionesMasEnsamblados(cadenaMontaje);
        if (conf.isEmpty()) {
            dashboard.update("Aún no hay configuraciones ensambladas.");
        } else {
            dashboard.update("Configuraciones más ensambladas:");
            for (Map.Entry<String, Integer> e : conf.entrySet()) {
                System.out.println(" - " + e.getKey() + " => " + e.getValue() + " uds.");
            }
        }
        break;
    }
    case 8: {
        sc.nextLine();
        System.out.print("Fecha (dd/MM/yyyy) — vacío = hoy: ");
        String fechaStr = sc.nextLine().trim();
        Date fecha;
        try {
            fecha = fechaStr.isEmpty()? new Date() : new
SimpleDateFormat("dd/MM/yyyy").parse(fechaStr);
        } catch (Exception ex) {
            dashboard.mostrarError("Fecha no válida. Usa el formato dd/MM/yyyy.");
            break;
        }
        List<RegistroMontaje> regs = sistemaGestion.consultarHistorialFecha(fecha);
        if (regs.isEmpty()) {
            dashboard.update("Sin registros para esa fecha.");
        } else {
            SimpleDateFormat fmt = new SimpleDateFormat("HH:mm:ss");
            dashboard.update("Historial de operaciones para " + new
SimpleDateFormat("dd/MM/yyyy").format(fecha) + ":");
            for (RegistroMontaje r : regs) {
                System.out.println(" - " + fmt.format(r.getFecha())
                    + " | " + r.getTipoComponente()
                    + " | " + r.getAccion());
            }
        }
        break;
    }
    case 0:
        System.out.println("Volviendo al menú principal...");
        break;
    default:
        System.out.println("Opción no válida.");
}
}

```

Practica de Programación Orientada a Objeto – Curs 2025-2026

```
}  
}
```